1.0 CPU

1.1 Features

- ARM Cortex-M4F
- FPU (Floating point unit) IEEE754 compliant
- ARMv7-M Thumb Instruction set
- 32-bit Harvard Architecture with Instruction and Data buses
- Little-Endian
- Unified address map with byte addressing
- Extended DSP instruction set with 32 bit Hardware Divider
- Nested Vectored Interrupt Controller(NVIC), low latency and configurable priority
- Memory Protection Unit
- 2 wire serial debug interfaces (SWD)
- 8 level of interrupt priority
- 2 break points and 1 watchdog and data value matching
- Bit-banding feature
- System Timer (SysTick)

2.0 BUS ARCHITECTURE

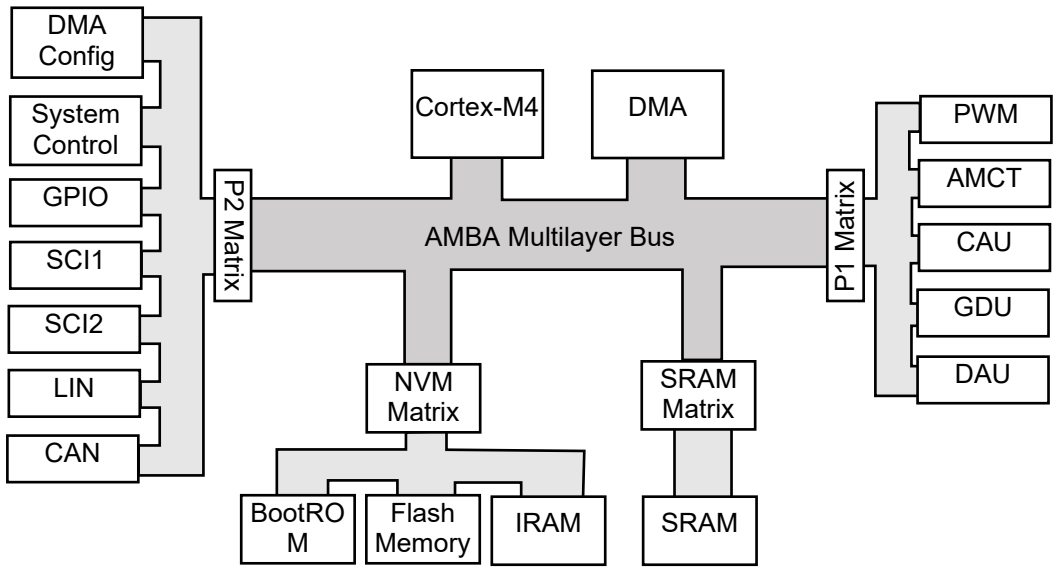
2.1 Multilayer Bus architecture

89201 uses 32bit width AMBA-lite bus to network the hardware. The peripherals are connected using AMBA Advanced Peripheral Bus (APB).

89201 bus configured as multi-layer bus structure. That allows to have 2 master nodes accessing to different peripheral at the same time. If both master node access to same matrix at the same time, bus arbitration will occur and low priory node will be on hold until high priority node transaction completed. For example, when CPU access to DRAM, DMA can access to a peripheral at the same time, both accesses will be executed without having any timing penalty. This will allow to have a deterministic execution period.

2.2 Bus Architecture

High level bus connection summary is shown as below.



2.3 Address Map

Detailed address map is as follows:

Address	Size	Matrix	Description
0x0000_0000 0x0000_7FFF	32kB	NVM	Boot ROM
0x0001_0000 0x0001_1FFF	8kB	NVM	Instruction RAM
0x0800_0000 0x0803_FFFF	Up to 256kB (Depending on variant)	NVM	Flash Memory
0x2000_0000 0x2000_7FFF	32kB	SRAM	Data RAM
0x4000_0000 0x4000_FFFF	64kB	P1	APB Peripheral Region
0x4001_0000 0x4001_FFFF	64kB	P1	PWM generator
0x4002_0000 0x4002_FFFF	64kB	P1	AMCT
0x4004_0000 0x4004_FFFF	64kB	P2	CAN
0x4007_0000 0x4007_FFFF	64kB	P2	DMA
0x4200_0000 0x4FFF_FFFF	16MB	BB	Peripheral Bit band region. No access from DMA. Only for P1 and P2.
0xE000_0000 0xE00F_FFFF	1MB	SYS	ARM CPU control

Note: Non listed area is unpopuldated. Access to those area will incur the bus fault.

APB Peripheral address map is listed as follows:

Address	Size	Matrix	Description
0x4000_0000 0x4000_0FFF	4kB	P0	CAU (Current Acquisition Unit)
0x4000_1000 0x4000_1FFF	4kB	P0	I2C GDU Master
0x4000_2000 0x4000_2FFF	4kB	P0	DAU (Data Acquisition Unit)
0x4000_8000 0x4000_8FFF	4kB	P1	GPIO
0x4000_9000 0x4000_9FFF	4kB	P1	GTU (General Timer Units)
0x4000_A000 0x4000_AFFF	4kB	P1	SCI1 (UART1/SPI1)
0x4000_B000 0x4000_BFFF	4kB	P1	SCI2 (UART2/SPI2)
0x4000_C000 0x4000_CFFF	4kB	P1	LIN
0x4000_D000 0x4000_DFFF	4kB	P1	System Control Unit

Note1: Non listed area is unpopulated. Access to those area will incur the bus fault.
Note2: All the peripheral accesses should be half(16-bit) or full(32-bit) word. Any other access type may result in unexpected behaviour.

2.4 Bus Access Arbitration

A89201 has 2 bus master, which are DMA and CPU. The bus arbitration has fixed priority. DMA has highest priority so if when DMA and CPU access to same bus slave node at the same time, DMA will be granted

mastership. The shared slots are described in section 2.3. The bus arbitration will only happen when more than 1 master access to a same matrix.

2.5 Bus Error

If the user access a non-populed AHB or APC address, the bus return bus fault. It will incur the bus fault IRQ to CPU. For peripherals, please refer each block for the bus error.

3.0 CPU FEATURE

3.1 Instruction Set summary

The processor implements the ARMv7-M Thumb instruction set. Table shows the Cortex-M4 instructions and their cycle counts. The cycle counts are based on a system with zero wait states.

Within the assembler syntax, depending on the operation, the <op2> field can be replaced with one of the following options:

a simple register specifier, for example Rm

an immediate shifted register, for example Rm, LSL #4

a register shifted register, for example Rm, LSL Rs

an immediate value, for example #0xE000E000.

For brevity, not all load and store addressing modes are shown. See the ARMv7-M Architecture Reference Manual for more information.

Table 3.1 uses the following abbreviations in the Cycles column:

P

The number of cycles required for a pipeline refill. This ranges from 1 to 3 depending on the alignment and width of the target instruction, and whether the processor manages to speculate the address early.

B

The number of cycles required to perform the barrier operation. For DSB and DMB, the minimum number of cycles is zero. For ISB, the minimum number of cycles is equivalent to the number required for a pipeline refill.

N

The number of registers in the register list to be loaded or stored, including PC or LR.

W

The number of cycles spent waiting for an appropriate event.

Operation	Description	Assembler	Cycles
Move	Register	MOV Rd, <op2>	1
	16-bit immediate	MOVW Rd, #<imm>	1
	Immediate into top	MOVT Rd, #<imm>	1
	To PC	MOV PC, Rm	1 + P
Add	Add	ADD Rd, Rn, <op2>	1
	Add to PC	ADD PC, PC, Rm	1 + P
	Add with carry	ADC Rd, Rn, <op2>	1
	Form address	ADR Rd, <label>	1
Subtract	Subtract	SUB Rd, Rn, <op2>	1
	Subtract with borrow	SBC Rd, Rn, <op2>	1
	Reverse	RSB Rd, Rn, <op2>	1
Multiply	Multiply	MUL Rd, Rn, Rm	1
	Multiply accumulate	MLA Rd, Rn, Rm	2
	Multiply subtract	MLS Rd, Rn, Rm	2
	Long signed	SMULL RdLo, RdHi, Rn, Rm	1
	Long unsigned	UMULL RdLo, RdHi, Rn, Rm	1
	Long signed accumulate	SMLAL RdLo, RdHi, Rn, Rm	1

	Long unsigned accumulate	UMLAL RdLo, RdHi, Rn, Rm	1
Divide	Signed	SDIV Rd, Rn, Rm	2 to 12[a]
	Unsigned	UDIV Rd, Rn, Rm	2 to 12[a]
Saturate	Signed	SSAT Rd, #<imm>, <op2>	1
	Unsigned	USAT Rd, #<imm>, <op2>	1
Compare	Compare	CMP Rn, <op2>	1
	Negative	CMN Rn, <op2>	1
Logical	AND	AND Rd, Rn, <op2>	1
	Exclusive OR	EOR Rd, Rn, <op2>	1
	OR	ORR Rd, Rn, <op2>	1
	OR NOT	ORN Rd, Rn, <op2>	1
	Bit clear	BIC Rd, Rn, <op2>	1
	Move NOT	MVN Rd, <op2>	1
	AND test	TST Rn, <op2>	1
	Exclusive OR test	TEQ Rn, <op1>	
Shift	Logical shift left	LSL Rd, Rn, #<imm>	1
	Logical shift left	LSL Rd, Rn, Rs	1
	Logical shift right	LSR Rd, Rn, #<imm>	1
	Logical shift right	LSR Rd, Rn, Rs	1
	Arithmetic shift right	ASR Rd, Rn, #<imm>	1
	Arithmetic shift right	ASR Rd, Rn, Rs	1
Rotate	Rotate right	ROR Rd, Rn, #<imm>	1
	Rotate right	ROR Rd, Rn, Rs	1
	With extension	RRX Rd, Rn	1
Count	Leading zeroes	CLZ Rd, Rn	1
Load	Word	LDR Rd, [Rn, <op2>]	2[b]
	To PC	LDR PC, [Rn, <op2>]	2[b] + P
	Halfword	LDRH Rd, [Rn, <op2>]	2[b]
	Byte	LDRB Rd, [Rn, <op2>]	2[b]
	Signed halfword	LDRSH Rd, [Rn, <op2>]	2[b]
	Signed byte	LDRSB Rd, [Rn, <op2>]	2[b]
	User word	LDRT Rd, [Rn, #<imm>]	2[b]
	User halfword	LDRHT Rd, [Rn, #<imm>]	2[b]
	User byte	LDRBT Rd, [Rn, #<imm>]	2[b]
	User signed halfword	LDRSHT Rd, [Rn, #<imm>]	2[b]
	User signed byte	LDRSBT Rd, [Rn, #<imm>]	2[b]
	PC relative	LDR Rd,[PC, #<imm>]	2[b]
	Doubleword	LDRD Rd, Rd, [Rn, #<imm>]	1 + N
	Multiple	LDM Rn, {<reglist>}	1 + N
	Multiple including PC	LDM Rn, {<reglist>, PC}	1 + N + P
Store	Word	STR Rd, [Rn, <op2>]	2[b]
	Halfword	STRH Rd, [Rn, <op2>]	2[b]
	Byte	STRB Rd, [Rn, <op2>]	2[b]
	Signed halfword	STRSH Rd, [Rn, <op2>]	2[b]
	Signed byte	STRSB Rd, [Rn, <op2>]	2[b]
	User word	STRT Rd, [Rn, #<imm>]	2[b]
	User halfword	STRHT Rd, [Rn, #<imm>]	2[b]
	User byte	STRBT Rd, [Rn, #<imm>]	2[b]
	User signed halfword	STRSHT Rd, [Rn, #<imm>]	2[b]
	User signed byte	STRSBT Rd, [Rn, #<imm>]	2b
	Doubleword	STRD Rd, Rd, [Rn, #<imm>]	1 + N
	Multiple	STM Rn, {<reglist>}	1 + N
Push	Push	PUSH {<reglist>}	1 + N
	Push with link register	PUSH {<reglist>, LR}	1 + N
Pop	Pop	POP {<reglist>}	1 + N

	Pop and return	POP {<reglist>, PC}	1 + N + P
Semaphore	Load exclusive	LDREX Rd, [Rn, #<imm>]	2
	Load exclusive half	LDREXH Rd, [Rn]	2
	Load exclusive byte	LDREXB Rd, [Rn]	2
	Store exclusive	STREX Rd, Rt, [Rn, #<imm>]	2
	Store exclusive half	STREXH Rd, Rt, [Rn]	2
	Store exclusive byte	STREXB Rd, Rt, [Rn]	2
	Clear exclusive monitor	CLREX	1
Branch	Conditional	B<cc> <label>	1 or 1 + P[c]
	Unconditional	B <label>	1 + P
	With link	BL <label>	1 + P
	With exchange	BX Rm	1 + P
	With link and exchange	BLX Rm	1 + P
	Branch if zero	CBZ Rn, <label>	1 or 1 + P[c]
	Branch if non-zero	CBNZ Rn, <label>	1 or 1 + P[c]
	Byte table branch	TBB [Rn, Rm]	2 + P
	Halfword table branch	TBH [Rn, Rm, LSL#1]	2 + P
State change	Supervisor call	SVC #<imm>	-
	If-then-else	IT... <cond>	1[d]
	Disable interrupts	CPSID <flags>	1 or 2
	Enable interrupts	CPSIE <flags>	1 or 2
	Read special register	MRS Rd, <specreg>	1 or 2
	Write special register	MSR <specreg>, Rn	1 or 2
	Breakpoint	BKPT #<imm>	-
Extend	Signed halfword to word	SXTH Rd, <op2>	1
	Signed byte to word	SXTB Rd, <op2>	1
	Unsigned halfword	UXTH Rd, <op2>	1
	Unsigned byte	UXTB Rd, <op2>	1
Bit field	Extract unsigned	UBFX Rd, Rn, #<imm>, #<imm>	1
	Extract signed	SBFX Rd, Rn, #<imm>, #<imm>	1
	Clear	BFC Rd, Rn, #<imm>, #<imm>	1
	Insert	BFI Rd, Rn, #<imm>, #<imm>	1
Reverse	Bytes in word	REV Rd, Rm	1
	Bytes in both halfwords	REV16 Rd, Rm	1
	Signed bottom halfword	REVSH Rd, Rm	1
	Bits in word	RBIT Rd, Rm	1
Hint	Send event	SEV	1
	Wait for event	WFE	1 + W
	Wait for interrupt	WFI	1 + W
	No operation	NOP	1
Barriers	Instruction synchronization	ISB	1 + B
	Data memory	DMB	1 + B
	Data synchronization	DSB <flags>	1 + B

[a] Division operations use early termination to minimize the number of cycles required based on the number of leading ones and zeroes in the input operands.

[b] Neighboring load and store single instructions can pipeline their address and data phases. This enables these instructions to complete in a single execution cycle.

[c] Conditional branch completes in a single cycle if the branch is not taken.

[d] An IT instruction can be folded onto a preceding 16-bit Thumb instruction, enabling execution in zero cycles.

Following shows the DSP instruction set.

Operation	Description	Assembler	Cycles

Multiply	32-bit multiply with 32-most-significant-bit accumulate	SMMLA	1
	32-bit multiply with 32-most-significant-bit subtract	SMMLS	1
	32-bit multiply returning 32-most-significant-bits	SMMUL	1
	32-bit multiply with rounded 32-most-significant-bit accumulate	SMMLAR	1
	32-bit multiply with rounded 32-most-significant-bit subtract	SMMLSR	1
	32-bit multiply returning rounded 32-most-significant-bits	SMMULR	1
Signed Multiply	Q setting 16-bit signed multiply with 32-bit accumulate, bottom by bottom	SMLABB	1
	Q setting 16-bit signed multiply with 32-bit accumulate, bottom by top	SMLABT	1
	16-bit signed multiply with 64-bit accumulate, bottom by bottom	SMLALBB	1
	16-bit signed multiply with 64-bit accumulate, bottom by top	SMLALBT	1
	Dual 16-bit signed multiply with single 64-bit accumulator	SMLALD	1
	16-bit signed multiply with 64-bit accumulate, top by bottom	SMLALTB	1
	16-bit signed multiply with 64-bit accumulate, top by top	SMLALTT	1
	16-bit signed multiply yielding 32-bit result, bottom by bottom	SMULBB	1
	16-bit signed multiply yielding 32-bit result, bottom by top	SMULBT	1
	16-bit signed multiply yielding 32-bit result, top by bottom	SMULTB	1
	16-bit signed multiply yielding 32-bit result, top by bottom	SMULTT	1
	16-bit by 32-bit signed multiply returning 32-most-significant-bits, bottom	SMULWB	1
	16-bit by 32-bit signed multiply returning 32-most-significant-bits, top	SMULWT	1
	Dual 16-bit signed multiply returning difference	SMUSD	1
	Q setting 16-bit signed multiply with 32-bit accumulate, top by bottom	SMLATB	1
	Q setting 16-bit signed multiply with 32-bit accumulate, top by top	SMLATT	1
	Q setting dual 16-bit signed multiply with single 32-bit accumulator	SMLAD	1
	Q setting 16-bit by 32-bit signed multiply with 32-bit accumulate, bottom	SMLAWB	1
	Q setting 16-bit by 32-bit signed multiply with 32-bit accumulate, top	SMLAWT	1
	Q setting dual 16-bit signed multiply subtract with 32-bit accumulate	SMLSD	1
	Q setting dual 16-bit signed multiply subtract with 64-bit accumulate	SMLSLD	1
	Q setting sum of dual 16-bit signed multiply	SMUAD	1
	Q setting pre-exchanged dual 16-bit signed multiply with single 32-bit accumulator	SMLADX	1
Unsigned Multiply	32-bit unsigned multiply with double 32-bit accumulation yielding 64-bit result	UMAAL	1
Saturate	Q setting dual 16-bit saturate	SSAT16	1
	Q setting dual 16-bit unsigned saturate	USAT16	1
Packing and Unpacking	Pack half word top with shifted bottom	PKHTB	1
	Pack half word bottom with shifted top	PKHBT	1
	Extract 8-bits and sign extend to 32-bits	SXTB	1

	Dual extract 8-bits and sign extend each to 16-bits	SXTB16	1
	Extract 16-bits and sign extend to 32-bits	SXTH	1
	Extract 8-bits and zero-extend to 32-bits	UXTB	1
	Dual extract 8-bits and zero-extend to 16-bits	UXTB16	1
	Extract 16-bits and zero-extend to 32-bits	UXTH	1
	Extract 8-bit to 32-bit unsigned addition	UXTAB	1
	Dual extracted 8-bit to 16-bit unsigned addition	UXTAB16	1
	Extracted 16-bit to 32-bit unsigned addition	UXTAH	1
	Extracted 8-bit to 32-bit signed addition	SXTAB	1
	Dual extracted 8-bit to 16-bit signed addition	SXTAB16	1
	Extracted 16-bit to 32-bit signed addition	SXTAH	1
Miscellaneous Data Processing	Select bytes based on GE bits	SEL	1
	Unsigned sum of quad 8-bit unsigned absolute difference	USAD8	1
	Unsigned sum of quad 8-bit unsigned absolute difference with 32-bit accumulate	USADA8	1
Addition	Dual 16-bit unsigned saturating addition	UQADD16	1
	Quad 8-bit unsigned saturating addition	UQADD8	1
	Q setting saturating add	QADD	1
	Q setting dual 16-bit saturating add	QADD16	1
	Q setting quad 8-bit saturating add	QADD8	1
	Q setting saturating double and add	QDADD	1
	GE setting quad 8-bit signed addition	SADD8	1
	GE setting dual 16-bit signed addition	SADD16	1
	Dual 16-bit signed addition with halved results	SHADD16	1
	Quad 8-bit signed addition with halved results	SHADD8	1
	GE setting dual 16-bit unsigned addition	UADD16	1
	GE setting quad 8-bit unsigned addition	UADD8	1
	Dual 16-bit unsigned addition with halved results	UHADD16	1
	Quad 8-bit unsigned addition with halved results	UHADD8	1
Subtraction	Q setting saturating double and subtract	QDSUB	1
	Dual 16-bit unsigned saturating subtraction	UQSUB16	1
	Quad 8-bit unsigned saturating subtraction	UQSUB8	1
	Q setting saturating subtract	QSUB	1
	Q setting dual 16-bit saturating subtract	QSUB16	1
	Q setting quad 8-bit saturating subtract	QSUB8	1
	Dual 16-bit signed subtraction with halved results	SHSUB16	1
	Quad 8-bit signed subtraction with halved results	SHSUB8	1
	GE setting dual 16-bit signed subtraction	SSUB16	1
	GE setting quad 8-bit signed subtraction	SSUB8	1
	Dual 16-bit unsigned subtraction with halved results	UHSUB16	1
	Quad 8-bit unsigned subtraction with halved results	UHSUB8	1
	GE setting dual 16-bit unsigned subtract	USUB16	1
	GE setting quad 8-bit unsigned subtract	USUB8	1
Parallel Addition and Subtraction	Dual 16-bit unsigned saturating addition and subtraction with exchange	UQASX	1
	Dual 16-bit unsigned saturating subtraction and addition with exchange	UQSAX	1
	GE setting dual 16-bit addition and subtraction with exchange	SASX	1
	Q setting dual 16-bit add and subtract with exchange	QASX	1

	Q setting dual 16-bit subtract and add with exchange	QSAX	1
	Dual 16-bit signed addition and subtraction with halved results	SHASX	1
	Dual 16-bit signed subtraction and addition with halved results	SHSAX	1
	GE setting dual 16-bit signed subtraction and addition with exchange	SSAX	1
	GE setting dual 16-bit unsigned addition and subtraction with exchange	UASX	1
	Dual 16-bit unsigned addition and subtraction with halved results and exchange	UHASX	1
	Dual 16-bit unsigned subtraction and addition with halved results and exchange	UHSAX	1
	GE setting dual 16-bit unsigned subtract and add with exchange	USAX	1

For more detailed information of instruction sets, please refer the ARM CortexM4 Technical Reference Manual.

3.2 Nester Vectored Interrupt Controller (NVIC)

The NVIC has 20 peripheral interrupt with up to 8 levels of priority. The priority of an interrupt can be changed dynamically. The NVIC and the processor core interface are closely coupled, to enable low latency interrupt processing and efficient processing of late arriving interrupts. The NVIC maintains knowledge of the stacked, or nested, interrupts to enable tail-chaining of interrupts.

The NVIC can only be fully accessed from privileged mode, but user can cause interrupts to enter a pending state in user mode if user enable the Configuration Control Register. Any other user mode access causes a bus fault.

All NVIC registers using byte, halfword, and word accesses shall be used unless otherwise stated.

All NVIC registers and system debug registers are little-endian.

3.3 Cotex-M4 Control register

Address	Core peripheral	Description
0xE000_E008 - 0xE000_E00F	System Control Block	See section 3.3.1.
0xE000_E010 - 0xE000_E01F	System timer	See section 3.3.2
0xE000_E100 - 0xE000_E4EF	NestedVectoredInterrupt Controller	See section 3.3.3
0xE000_ED00 - 0xE000_ED3F	System Control Block	See section 3.3.1
0xE000_ED90 - 0xE000_ED93	MPU Type Register	
0xE000_ED90 - 0xE000_EDB8	Memory Protection Unit	See section 3.3.4
0xE000_EF00 - 0xE000_EF03	Nested Vectored Interrupt Controller	
0xE000_EF30 - 0xE000_EF44	Floating Point Unit	

3.3.1 System Control Block

Address	Name	Type	Required Previlege	Reset Value
0xE000_E008	ACTLR	RW	Privileged	0x0000_0000
0xE000_ED00	CPUID	RO	Privileged	0x410F_C240
0xE000_ED04	ICSR	RW[a]	Privileged	0x0000_0000
0xE000_ED08	VTOR	RW	Privileged	0x0000_0000
0xE000_ED0C	AIRCR	RW[a]	Privileged	0xFA05_0000
0xE000_ED10	SCR	RW	Privileged	0x0000_0000
0xE000_ED14	CCR	RW	Privileged	0x0000_0200
0xE000_ED18	SHPR1	RW	Privileged	0x0000_0000
0xE000_ED1C	SHPR2	RW	Privileged	0x0000_0000
0xE000_ED20	SHPR3	RW	Privileged	0x0000_0000
0xE000_ED24	SHCRS	RW	Privileged	0x0000_0000

0xE000_ED28	CFSR	RW	Privileged	0x0000_0000
0xE000_ED28	MMSR[b]	RW	Privileged	0x00
0xE000_ED29	BFSR[b]	RW	Privileged	0x00
0xE000_ED2A	UFSR[b]	RW	Privileged	0x0000
0xE000_ED2C	HFSR	RW	Privileged	0x0000_0000
0xE000_ED34	MMAR	RW	Privileged	Unknown
0xE000_ED38	BFAR	RW	Privileged	Unknown
0xE000_ED3C	AFSR	RW	Privileged	0x0000_0000

a. See the register description for more information.

b. A subregister of the CFSR.

Auciliary Control Register (ACTLR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E008

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	DISOOFP	DISFPCA
U	U	U	U	U	U	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	-	-	DISFOLD	DISDEFWBUF	DISMCYCINT
U	U	U	U	U	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
DISMCYCINT	0	When set to 1, disables interruption of load multiple and store multiple instructions. This increases the interrupt latency of the processor because any LDM or STM must complete before the processor can stack the current state and enter the interrupt handler.
DISDEFWBUF	1	When set to 1, disables write buffer use during default memory map accesses. This causes all BusFaults to be precise BusFaults but decreases performance because any store to memory must complete before the processor can execute the next instruction. Note: This bit only affects write buffers implemented in the Cortex-M4 processor.
DISFOLD	2	When set to 1, disables IT folding. In some situations, the processor can start executing the first instruction in an IT block while it is still executing the IT instruction. This behavior is called IT folding, and improves performance, However, IT folding can cause jitter in

		looping. If a task must avoid jitter, set the DISFOLD bit to 1 before executing the task, to disable IT folding.
DISFPCA	8	Disables automatic update of CONTROL.FPCA.
DISOOFP	9	Disables floating point instructions completing out of order with respect to integer instructions.

CPUID Base Register

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED00

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
Implementer[7:0]							
R							

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
Variant[3:0]				Constant[3:0]			
R				R			

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
PartNo[11:4]							
R							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
PartNo[3:0]				Revision[3:0]			
R				R			

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
Revision	3:0	Revision number, the p value in the rnpn product revision identifier: 0x0 = Patch 0
PartNo	15:4	Part number of the processor: 0xC24 = Cortex-M4
Constant	19:16	Reads as 0xF
Variant	23:20	Variant number, the r value in the rnpn product revision identifier: 0x0 = Revision 0
Implementer	31:24	Implementer code: 0x41 = ARM

Interrupt Control and State Register (ISCR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED04

The ICSR:

provides:

— a set-pending bit for the Non-Maskable Interrupt (NMI) exception

— set-pending and clear-pending bits for the PendSV and SysTick exceptions

indicates:

— the exception number of the exception being processed

— whether there are preempted active exceptions

— the exception number of the highest priority pending exception

— whether any interrupts are pending

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
NMIPENDSET	-	-	PENDSVSET	PENDSVCLR	PENDSTSET	PENDSTCLR	-
R/W	U	U	R/W	W	R/W	W	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
Reserved for Debug use	ISRPENDING	-	-	-	-	VECTPENDING	
R	R	U	U	U	U	R	R

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
VECTPENDING				RETTOBASE	-	-	VECTACTIVE
U	U	U	U	U	U	R/W	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
VECTACTIVE							
R	R	R	R	R	R	R	R

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
VECTACTIVE	8:0	Contains the active exception number: 0 = Thread mode Nonzero = The exception numbera of the currently active exception.
RETTOBASE	11	Indicates whether there are preempted active exceptions: 0 = there are preempted active exceptions to execute 1 = there are no active exceptions, or the currently-executing exception is the only active exception
VECTPENDING	17:12	Indicates the exception number of the highest priority pending enabled exception: 0 = no pending exceptions Nonzero = the exception number of the highest priority pending enabled exception. The value indicated by this field includes the effect of the BASEPRI and FAULTMASK registers, but not any effect of the PRIMASK register.
ISRPENDING	22	Interrupt pending flag, excluding NMI and Faults: 0 = interrupt not pending 1 = interrupt pending.
Reserved for Debug use	23	This bit is reserved for Debug use and reads-as-zero when the processor is not in Debug
PENDSTCLR	25	SysTick exception clear-pending bit. Write: 0 = no effect 1 = removes the pending state from the SysTick exception. This bit is WO. On a register read its value is Unknown.

PENDSTSET	26	SysTick exception set-pending bit. Write: 0 = no effect 1 = changes SysTick exception state to pending. Read: 0 = SysTick exception is not pending 1 = SysTick exception is pending.
PENDSVCLR	27	PendSV clear-pending bit. Write: 0 = no effect 1 = removes the pending state from the PendSV exception.
PENDSVSET	28	PendSV set-pending bit. Write: 0 = no effect 1 = changes PendSV exception state to pending. Read: 0 = PendSV exception is not pending 1 = PendSV exception is pending. Writing 1 to this bit is the only way to set the PendSV exception state to pending
NMIPENDSET	31	NMI set-pending bit. Write: 0 = no effect 1 = changes NMI exception state to pending. Read: 0 = NMI exception is not pending 1 = NMI exception is pending. Because NMI is the highest-priority exception, normally the processor enter the NMI exception handler as soon as it registers a write of 1 to this bit, and entering the handler clears this bit to 0. A read of this bit by the NMI exception handler returns 1 only if the NMI signal is reasserted while the processor is executing that handler.

Vector Table Offset Register (VTOR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED08

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
TBLOFF[24:17]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
TBLOFF[16:9]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
TBLOFF[8:1]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
TBLOFF[0]	-	-	-	-	-	-	-
R/W	U	U	U	U	U	U	U

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
TBLOFF	31:7	Vector table base offset field. It contains bits[29:7] of the offset of the table base from the bottom of the memory map. Note Bit[29] determines whether the vector table is in the code or SRAM memory region: • 0 = code • 1 = SRAM.

In implementations bit[29] is sometimes called the TBLBASE bit.

Application Interrupt and Reset Control Register (AIRCR)

Reset value = 0x0000_0000 RegisterAddress:0xE000ED0C

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
			Write: VECTKEYSTAT Read: VECTKEY				
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
			Write: VECTKEYSTAT Read: VECTKEY				
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
ENDIANNESS	-	-	-	-	PRIGROUP		
R	U	U	U	U	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	-	-	SYSRESETREQ	VECTCLRACTIVE	VECTRESET
U	U	U	U	U	W	W	W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
VECTRESET	0	Reserved for Debug use. This bit reads as 0. When writing to the register you must write 0 to this bit, otherwise behavior is Unpredictable.
VECTCLRACTIVE	1	Reserved for Debug use. This bit reads as 0. When writing to the register you must write 0 to this bit, otherwise behavior is Unpredictable.
SYSRESETREQ	2	System reset request bit is implementation defined: 0 = no system reset request 1 = asserts a signal to the outer system that requests a reset. This is intended to force a large system reset of all major components except for debug. This bit reads as 0. See you vendor documentation for more information about the use of this signal in your implementation.
PRIGROUP	10:8	Interrupt priority grouping field is implementation defined. This field determines the split of group priority from subpriority. See the binary point section below.
ENDIANNESS	15	Data endianness bit is implementation defined: 0 = Little-endian 1 = Big-endian.
Write: VECTKEYSTAT Read: VECTKEY	31:16	Register key: Reads as 0xFA05 On writes, write 0x5FA to VECTKEY, otherwise the write is ignored.

Binary point

The PRIGROUP field indicates the position of the binary point that splits the PRI_n fields in the Interrupt Priority Registers into separate group priority and subpriority fields. Table 4-18 shows how the PRIGROUP value controls this split. Implementations having fewer than 8-bits of interrupt priority treat the least significant bits as zero.

	Interrupt priority level value, PRI_N[7:0]			Number of	
PRIGROUP	Binary point[a]	Group priority bits	Subpriority bits	Group priorities	Subpriorities
0b000	bxxxxxxx.y	[7:1]	[0]	128	2
0b001	bxxxxxx.yy	[7:2]	[1:0]	64	4

0b010	bxxxxx.yyy	[7:3]	[2:0]	32	8
0b011	bxxxx.yyyy	[7:4]	[3:0]	16	16
0b100	bxxx.yyyyy	[7:5]	[4:0]	8	32
0b101	bxx.yyyyyy	[7:6]	[5:0]	4	64
0b110	bx.yyyyyyy	[7]	[6:0]	2	128
0b111	b.yyyyyyyy	None	[7:0]	1	256

a. PRI_n[7:0] field showing the binary point. x denotes a group priority field bit, and y denotes a subpriority field bit.

System Control Register (SCR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED10

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U		

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	SEVONPEND	-	SLEEPDEEP	SLEEPONEXIT	-
U	U	U	R/W	U	R/W	R/W	U

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
SLEEPONEXIT	1	Indicates sleep-on-exit when returning from Handler mode to Thread mode: 0 = do not sleep when returning to Thread mode. 1 = enter sleep, or deep sleep, on return from an ISR. Setting this bit to 1 enables an interrupt driven application to avoid returning to an empty main application.
SLEEPDEEP[*1]	2	Controls whether the processor uses sleep or deep sleep as its low power mode: 0 = sleep 1 = deep sleep.
SEVONPEND	4	Send Event on Pending bit: 0 = only enabled interrupts or events can wakeup the processor, disabled interrupts are excluded 1 = enabled events and all interrupts, including disabled interrupts, can wakeup the processor. When an event or interrupt enters pending state, the event signal wakes up the processor from WFE. If the processor is not waiting for an event, the event is registered and affects the next WFE. The processor also wakes up on execution of an SEV instruction or an external event.

*1: Sleep mode is not implemented. No effect by enabling this feature.

Configuration and Control Register (CCR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED14

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	STKALIGN	BFHFNMIGN
U	U	U	U	U	U	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	DIV_0_TRP	UNALIGN_TRP	-	USERSETMPEND	NONBASETHRDENA
U	U	U	R/W	R/W	U	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
NONBASETHRDENA	0	Indicates how the processor enters Thread mode: 0 = processor can enter Thread mode only when no exception is active. 1 = processor can enter Thread mode from any level under the control of an EXC_RETURN value.
USERSETMPEND	1	Enables unprivileged software access to the STIR, 0 = disable 1 = enable.
UNALIGN_TRP	3	Enables unaligned access traps: 0 = do not trap unaligned halfword and word accesses 1 = trap unaligned halfword and word accesses. If this bit is set to 1, an unaligned access generates a UsageFault. Unaligned LDM, STM, LDRD, and STRD instructions always fault irrespective of whether UNALIGN_TRP is set to 1.
DIV_0_TRP	4	Enables faulting or halting when the processor executes an SDIV or UDIV instruction with a divisor of 0: 0 = do not trap divide by 0 1 = trap divide by 0. When this bit is set to 0, a divide by zero returns a quotient of 0.

BFHFNMIGN	8	Enables handlers with priority -1 or -2 to ignore data BusFaults caused by load and store instructions. This applies to the hard fault, NMI, and FAULTMASK escalated handlers: 0 = data bus faults caused by load and store instructions cause a lock-up 1 = handlers running at priority -1 and -2 ignore data bus faults caused by load and store instructions. Set this bit to 1 only when the handler and its data are in absolutely safe memory. The normal use of this bit is to probe system devices and bridges to detect control path problems and fix them.
STKALIGN	9	Indicates stack alignment on exception entry: 0 = 4-byte aligned 1 = 8-byte aligned. On exception entry, the processor uses bit[9] of the stacked PSR to indicate the stack alignment. On return from the exception it uses this stacked bit to restore the correct stack alignment.

System Handler Priority Registers (SHPR1)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED18

The SHPR1-SHPR3 registers set the priority level, 0 to 255, of the exception handlers that have configurable priority. Note Valid priority level is up to 8.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
PRI_6[7:0]							
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
PRI_5[7:0]							
U	U	U	U	U	U		

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
PRI_4[7:0]							
U	U	U	U	U			

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
PRI_4	7:0	Priority of system handler 4, MemManage
PRI_5	15:8	Priority of system handler 5, BusFault
PRI_6	23:16	Priority of system handler 6, UsageFault

System Handler Priority Registers (SHPR2)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED1C

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
PRI_11[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
U	U	U	U	U	U	U	U

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
PRI_11	31:24	Priority of system handler 11, SVCall

System Handler Priority Registers (SHPR3)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED20

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
PRI_15							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
PRI_14							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8

U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
U	U	U	U	U	U	U	U

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
PRI_14	23:16	Priority of system handler 14, PendSV
PRI_15	31:24	Priority of system handler 15, SysTick exception

System Handler Control and State Register (SHCSR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED24

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
					USGFAULTENA	BUSFAULTENA	MEMFAULTENA
U	U	U	U	U	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
SVCALLPENDED	BUSFAULTPENDED	MEMFAULTPENDED	USGFAULTPENDED	SYSTICKACT	PENDSVACT		MONITORACT
R/W	R/W	R/W	R/W	R/W	R/W	U	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
SVCALLACT				USGFAULTACT		BUSFAULTACT	MEMFAULTACT
R/W	U	U	U	R/W	U	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
MEMFAULTACT	0	MemManage exception active bit, reads as 1 if exception is active
BUSFAULTACT	1	BusFault exception active bit, reads as 1 if exception is active
USGFAULTACT	3	UsageFault exception active bit, reads as 1 if exception is active
SVCALLACT	7	SVCall active bit, reads as 1 if SVC call is active
MONITORACT	8	Debug monitor active bit, reads as 1 if Debug monitor is active

PENDSVACT	10	PendSV exception active bit, reads as 1 if exception is active
SYSTICKACT	11	SysTick exception active bit, reads as 1 if exception is active[c]
USGFAULTPENDED	12	UsageFault exception pending bit, reads as 1 if exception is pending[b]
MEMFAULTPENDED	13	MemManage exception pending bit, reads as 1 if exception is pending[b]
BUSFAULTPENDED	14	BusFault exception pending bit, reads as 1 if exception is pending[b]
SVCALLPENDED	15	SVCall pending bit, reads as 1 if exception is pending[b]
MEMFAULTENA	16	MemManage enable bit, set to 1 to enable[a]
BUSFAULTENA	17	BusFault enable bit, set to 1 to enable[a]
USGFAULTENA	18	UsageFault enable bit, set to 1 to enable[a]

a. Enable bits, set to 1 to enable the exception, or set to 0 to disable the exception.
b. Pending bits, read as 1 if the exception is pending, or as 0 if it is not pending. You can write to these bits to change the pending status of the exceptions.
c. Active bits, read as 1 if the exception is active, or as 0 if it is not active. You can write to these bits to change the active status of the exceptions, but see the Caution in this section.

If you disable a system handler and the corresponding fault occurs, the processor treats the fault as a hard fault.
You can write to this register to change the pending or active status of system exceptions. An OS kernel can write to the active bits to perform a context switch that changes the currentexception type.

Caution

- Software that changes the value of an active bit in this register without correct adjustment to the stacked content can cause the processor to generate a fault exception. Ensure software that writes to this register retains and subsequently restores the current active status.
- After you have enabled the system handlers, if you have to change the value of a bit in this register you must use a read-modify-write procedure to ensure that you change only the required bit.

Configure Fault Status Register (CFSR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED28

The CFSR indicates the cause of a MemManage fault, BusFault, or UsageFault.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	DIVBYZERO	UNALIGNED
U	U	U	U	U	U	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	NOCP	INVPC	INVSTATE	UNDEFINSTR
U	U	U	U	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
BFARVALID		LSPERR	STKERR	UNSTKERR	IMPRECISERR	PRECISERR	IBUSERR
R/W	U	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
MMARVALID	-	MLSPERRa	MSTKERR	MUNSTKERR	-	DACCVIOL	IACCVIOL
R/W	U	R/W	R/W	R/W	U	R/W	R/W

Legend:

R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
IACCVIOL	0	Instruction access violation flag: 0 = no instruction access violation fault 1 = the processor attempted an instruction fetch from a location that does not permit execution. This fault occurs on any access to an XN region, even when the MPU is disabled or not present. When this bit is 1, the PC value stacked for the exception return points to the faulting instruction. The processor has not written a fault address to the MMAR.
DACCVIOL	1	Data access violation flag: 0 = no data access violation fault 1 = the processor attempted a load or store at a location that does not permit the operation. When this bit is 1, the PC value stacked for the exception return points to the faulting instruction. The processor has loaded the MMAR with the address of the attempted access.
MUNSTKERR	3	MemManage fault on unstacking for a return from exception: 0 = no unstacking fault 1 = unstack for an exception return has caused one or more access violations. This fault is chained to the handler. This means that when this bit is 1, the original return stack is still present. The processor has not adjusted the SP from the failing return, and has not performed a new save. The processor has not written a fault address to the MMAR.
MSTKERR	4	MemManage fault on stacking for exception entry: 0 = no stacking fault 1 = stacking for an exception entry has caused one or more access violations. When this bit is 1, the SP is still adjusted but the values in the context area on the stack might be incorrect. The processor has not written a fault address to the MMAR.
MLSPERRa	5	0 = no MemManage fault occurred during floating-point lazy state preservation 1 = a MemManage fault occurred during floating-point lazy state preservation.
MMARVALID	7	MemManage Fault Address Register (MMFAR) valid flag: 0 = value in MMAR is not a valid fault address 1 = MMAR holds a valid fault address. If a MemManage fault occurs and is escalated to a HardFault because of priority, the HardFault handler must set this bit to 0. This prevents problems on return to a stacked active MemManage fault handler whose MMAR value has been overwritten.
IBUSERR	8	Instruction bus error: 0 = no instruction bus error 1 = instruction bus error. The processor detects the instruction bus error on prefetching an instruction, but it sets the IBUSERR flag to 1 only if it attempts to issue the faulting instruction.

		When the processor sets this bit is 1, it does not write a fault address to the BFAR.
PRECISERR	9	Precise data bus error: 0 = no precise data bus error 1 = a data bus error has occurred, and the PC value stacked for the exception return points to the instruction that caused the fault. When the processor sets this bit is 1, it writes the faulting address to the BFAR.
IMPRECISERR	10	Imprecise data bus error: 0 = no imprecise data bus error 1 = a data bus error has occurred, but the return address in the stack frame is not related to the instruction that caused the error. When the processor sets this bit to 1, it does not write a fault address to the BFAR. This is an asynchronous fault. Therefore, if it is detected when the priority of the current process is higher than the BusFault priority, the BusFault becomes pending and becomes active only when the processor returns from all higher priority processes. If a precise fault occurs before the processor enters the handler for the imprecise BusFault, the handler detects both IMPRECISERR set to 1 and one of the precise fault status bits set to 1.
UNSTKERR	11	BusFault on unstacking for a return from exception: 0 = no unstacking fault 1 = unstack for an exception return has caused one or more BusFaults. This fault is chained to the handler. This means that when the processor sets this bit to 1, the original return stack is still present. The processor does not adjust the SP from the failing return, does not performed a new save, and does not write a fault address to the BFAR.
STKERR	12	BusFault on stacking for exception entry: 0 = no stacking fault 1 = stacking for an exception entry has caused one or more BusFaults. When the processor sets this bit to 1, the SP is still adjusted but the values in the context area on the stack might be incorrect. The processor does not write a fault address to the BFAR.
LSPERR[a]	13	0 = no bus fault occurred during floating-point lazy state preservation 1 = a bus fault occurred during floating-point lazy state preservation.
BFARVALID	15	BusFault Address Register (BFAR) valid flag: 0 = value in BFAR is not a valid fault address 1 = BFAR holds a valid fault address. The processor sets this bit to 1 after a BusFault where the address is known. Other faults can set this bit to 0, such as a MemManage fault occurring later. If a BusFault occurs and is escalated to a hard fault because of priority, the hard fault handler must set this bit to 0. This prevents problems if returning to a stacked active BusFault handler whose BFAR value has been overwritten.
UNDEFINSTR	16	Undefined instruction UsageFault: 0 = no undefined instruction UsageFault 1 = the processor has attempted to execute an undefined instruction.

		When this bit is set to 1, the PC value stacked for the exception return points to the undefined instruction. An undefined instruction is an instruction that the processor cannot decode
INVSTATE	17	Invalid state UsageFault: 0 = no invalid state UsageFault 1 = the processor has attempted to execute an instruction that makes illegal use of the EPSR. When this bit is set to 1, the PC value stacked for the exception return points to the instruction that attempted the illegal use of the EPSR. This bit is not set to 1 if an undefined instruction uses the EPSR.
INVPC	18	Invalid PC load UsageFault, caused by an invalid PC load by EXC_RETURN: 0 = no invalid PC load UsageFault 1 = the processor has attempted an illegal load of EXC_RETURN to the PC, as a result of an invalid context, or an invalid EXC_RETURN value. When this bit is set to 1, the PC value stacked for the exception return points to the instruction that tried to perform the illegal load of the PC.
NOCP	19	No coprocessor UsageFault. The processor does not support coprocessor instructions: 0 = no UsageFault caused by attempting to access a coprocessor 1 = the processor has attempted to access a coprocessor.
UNALIGNED	24	Unaligned access UsageFault: 0 = no unaligned access fault, or unaligned access trapping not enabled 1 = the processor has made an unaligned memory access. Enable trapping of unaligned accesses by setting the UNALIGN_TRP bit in the CCR to 1, see Configuration and Control Register. Unaligned LDM, STM, LDRD, and STRD instructions always fault irrespective of the setting of UNALIGN_TRP.
DIVBYZERO	25	Divide by zero UsageFault: 0 = no divide by zero fault, or divide by zero trapping not enabled 1 = the processor has executed an SDIV or UDIV instruction with a divisor of 0. When the processor sets this bit to 1, the PC value stacked for the exception return points to the instruction that performed the divide by zero. Enable trapping of divide by zero by setting the DIV_0_TRP bit in the CCR to 1.

a. Only present in a Cortex-M4F device.

UsageFault Status Register (UFSR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED2A

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16

U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
						DIVBYZERO	UNALIGNED
U	U	U	U	U	U	R/W 1C	R/W 1C

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
				NOCP	INVPC	INVSTATE	UNDEFINSTR
U	U	U	U	R/W 1C	R/W 1C	R/W 1C	R/W 1C

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
UNDEFINSTR	0	Undefined instruction UsageFault: 0 = no undefined instruction UsageFault 1 = the processor has attempted to execute an undefined instruction. When this bit is set to 1, the PC value stacked for the exception return points to the undefined instruction. An undefined instruction is an instruction that the processor cannot decode.
INVSTATE	1	Invalid state UsageFault: 0 = no invalid state UsageFault 1 = the processor has attempted to execute an instruction that makes illegal use of the EPSR. When this bit is set to 1, the PC value stacked for the exception return points to the instruction that attempted the illegal use of the EPSR. This bit is not set to 1 if an undefined instruction uses the EPSR.
INVPC	2	Invalid PC load UsageFault, caused by an invalid PC load by EXC_RETURN: 0 = no invalid PC load UsageFault 1 = the processor has attempted an illegal load of EXC_RETURN to the PC, as a result of an invalid context, or an invalid EXC_RETURN value. When this bit is set to 1, the PC value stacked for the exception return points to the instruction that tried to perform the illegal load of the PC.
NOCP	3	No coprocessor UsageFault. The processor does not support coprocessor instructions: 0 = no UsageFault caused by attempting to access a coprocessor 1 = the processor has attempted to access a coprocessor.
UNALIGNED	8	Unaligned access UsageFault: 0 = no unaligned access fault, or unaligned access trapping not enabled 1 = the processor has made an unaligned memory access. Enable trapping of unaligned accesses by setting the UNALIGN_TRP bit in the CCR to 1. Unaligned LDM, STM, LDRD, and STRD instructions always fault irrespective of the setting of UNALIGN_TRP.

DIVBYZERO	9	Divide by zero UsageFault: 0 = no divide by zero fault, or divide by zero trapping not enabled 1 = the processor has executed an SDIV or UDIV instruction with a divisor of 0. When the processor sets this bit to 1, the PC value stacked for the exception return points to the instruction that performed the divide by zero. Enable trapping of divide by zero by setting the DIV_0_TRP bit in the CCR to 1.

HardFault Status Register (HFSR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED2C

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
DEBUGEVT	FORCED						
R/W	R/W	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
						VECTTBL	
U	U	U	U	U	U	R/W	U

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
VECTTBL	1	Indicates a BusFault on a vector table read during exception processing: 0 = no BusFault on vector table read 1 = BusFault on vector table read. This error is always handled by the hard fault handler. When this bit is set to 1, the PC value stacked for the exception return points to the instruction that was preempted by the exception.
FORCED	30	Indicates a forced hard fault, generated by escalation of a fault with configurable priority that cannot be handles, either because of priority or because it is disabled: 0 = no forced HardFault 1 = forced HardFault.

		When this bit is set to 1, the HardFault handler must read the other fault status registers to find the cause of the fault.
DEBUGEVT	31	Reserved for Debug use. When writing to the register you must write 0 to this bit, otherwise behavior is Unpredictable.

MemManage Fault Address Register (MMFAR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED34

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
ADDRESS[31:24]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
ADDRESS[23:16]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
ADDRESS[15:8]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
ADDRESS[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
ADDRESS	31:0	When the MMARVALID bit of the MMFSR is set to 1, this field holds the address of the location that generated the MemManage fault

When an unaligned access faults, the address is the actual address that faulted. Because a single read or write instruction can be split into multiple aligned accesses, the fault address can be any address in the range of the requested access size. Flags in the MMFSR indicate the cause of the fault, and whether the value in the MMFAR is valid.

BusFault Address Register (BFAR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED38

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
ADDRESS[31:24]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16

ADDRESS[23:16]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
ADDRESS[15:8]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
ADDRESS[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
ADDRESS	31:0	When the BFARVALID bit of the BFSR is set to 1, this field holds the address of the location that generated the BusFault

When an unaligned access faults the address in the BFAR is the one requested by the instruction, even if it is not the address of the fault.
Flags in the BFSR indicate the cause of the fault, and whether the value in the BFAR is valid.

Auxiliary Fault Status Register (AFSR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED3C

The AFSR contains additional system fault information.

This register is read, write to clear. This means that bits in the register read normally, but writing 1 to any bit clears that bit to 0.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
IMPDEF[31:24]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
IMPDEF[23:16]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
IMPDEF[15:8]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IMPDEF[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
IMPDEF	31:0	Implementation defined. The bits map to the AUXFAULT input signals.

Each AFSR bit maps directly to an AUXFAULT input of the processor, and a single-cycle HIGH signal on the input sets the corresponding AFSR bit to one. It remains set to 1 until you write 1 to the bit to clear it to zero. See your vendor documentation for more information.
When an AFSR bit is latched as one, an exception does not occur. Use an interrupt if an exception is required.

3.3.2 System timer

The processor has a 24-bit system timer, SysTick, that counts down from the reload value to zero, reloads, that is wraps to, the value in the SYST_RVR register on the next clock edge, then counts down on subsequent clocks.

Note

When the processor is halted for debugging the counter does not decrement.

The system timer registers are:

Address	Name	Type	Required Previlege	Reset Value
0xE000_E010	SYST_CSR	RW	Privileged	*1
0xE000_E014	SYST_RVR	RW	Privileged	Unknown
0xE000_E018	SYST_CVR	RW	Privileged	Unknown
0xE000_E01C	SYST_CALIB	RO	Privileged	*1

a. See the register description for more information.

SysTick Control and Status Register (SYST_CSR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E010

The SysTick SYST_CSR register enables the SysTick features. The register resets to 0x00000000, or to 0x00000004 The A89201 does not use a reference clock but uses the CPU clock instead.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	-	-	-	COUNTFLAG
U	U	U	U	U	U	U	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	-	-	CLKSOURCE	TICKINT	ENABLE
U	U	U	U	U	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
ENABLE	0	Enables the counter: 0 = counter disabled 1 = counter enabled
TICKINT	1	Enables SysTick exception request: 0 = counting down to zero does not assert the SysTick exception request 1 = counting down to zero asserts the SysTick exception request. Software can use COUNTFLAG to determine if SysTick has ever counted to zero.
CLKSOURCE	2	Indicates the clock source: 0 = external clock 1 = processor clock.
COUNTFLAG	16	Returns 1 if timer counted to 0 since last time this was read.

When ENABLE is set to 1, the counter loads the RELOAD value from the SYST_RVR register and then counts down. On reaching 0, it sets the COUNTFLAG to 1 and optionally asserts the SysTick depending on the value of TICKINT. It then loads the RELOAD value again, and begins counting.

SysTick Reload Value Register (SYST_RVR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E014

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
RELOAD[23:16]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
RELOAD[15:8]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
RELOAD[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
RELOAD	23:0	Value to load into the SYST_CVR register when the counter is enabled and when it reaches 0

Calculating the RELOAD value

The RELOAD value can be any value in the range 0x00000001-0x00FFFFFF. A start value of 0 is possible, but has no effect because the SysTick exception request and COUNTFLAG are activated when counting from 1 to 0.
The RELOAD value is calculated according to its use. For example, to generate a multi-shot timer with a period of N processor clock cycles, use a RELOAD value of N-1. If the SysTick interrupt is required every 100 clock pulses, set RELOAD to 99.

SysTick Current Value Register (SYST_CVR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E018

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
CURRENT[23:16]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
CURRENT[15:8]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
CURRENT[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
CURRENT	23:0	Reads return the current value of the SysTick counter. A write of any value clears the field to 0, and also clears the SYST_CSR COUNTFLAG bit to 0.

SysTick Calibration Value Register (SYST_CALIB)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E01C

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
NOREF	SKEW						
R/W	R/W	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
TENMS[23:16]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
TENMS[15:8]							

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
TENMS[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
TENMS	23:0	Reload value for 10ms (100Hz) timing, subject to system clock skew errors. If the value reads as zero, the calibration value is not known.
SKEW	30	Indicates whether the TENMS value is exact: 0 = TENMS value is exact 1 = TENMS value is inexact, or not given. An inexact TENMS value can affect the suitability of SysTick as a software real time clock.
NOREF	31	Indicates whether the device provides a reference clock to the processor: 0 = reference clock provided 1 = no reference clock provided. If your device does not provide a reference clock, the SYST_CSR.CLKSOURCE bit reads-as-one and ignores writes.

If calibration information is not known, calculate the calibration value required from the frequency of the processor clock or external clock.

3.3.3 Nested Vectored Interrupt Controller

This section describes the NVIC and the registers it uses. The NVIC supports:

- An implementation-defined number of interrupts, in the range 1-240 interrupts.
- A programmable priority level of 0-255 for each interrupt. A higher level corresponds to a lower priority, so level 0 is the highest interrupt priority.
- Level and pulse detection of interrupt signals.
- Dynamic reprioritization of interrupts.
- Grouping of priority values into group priority and subpriority fields.
- Interrupt tail-chaining.
- An external Non Maskable Interrupt (NMI)

The processor automatically stacks its state on exception entry and unstacks this state on exception exit, with no instruction overhead. This provides low latency exception handling. The hardware implementation of the NVIC registers is:

Address	Name	Type	Required Previlege	Reset Value
0xE000_E100	NVIC_ISER	RW	Privileged	0x0000_0000
0XE000_E180	NVIC_ICER	RW	Privileged	0x0000_0000
0XE000_E200	NVIC_ISPR	RW	Privileged	0x0000_0000
0XE000_E280	NVIC_ICPR	RW	Privileged	0x0000_0000
0xE000_E300	NVIC_IABR	RW	Privileged	0x0000_0000
0xE000_E400- 0xE000_E410	NVIC_IPR0- NVIC_IPR4	RW	Privileged	0x0000_0000
0xE000_EF00	STIR	WO	Configurable[a]	0x0000_0000

a. See the register description for more information.

Interrupt Set-enable Registers (NVIC_ISER)

Reset value = 0x0000_0000 RegisterAddress: 0xE000_E100

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	IRQ_SysCon	IRQ_DMA	IRQ_AMCT	IRQ_GPIO
U	U	U	U	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
IRQ_CAN	IRQ_GDU_st	IRQ _Ig	IRQ_LIN	IRQ_SCI2	IRQ_SCI1	IRQ_GTU8	IRQ_GTU7
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IRQ_GTU6	IRQ_GTU5	IRQ_GTU4	IRQ_GTU3	IRQ_GTU2	IRQ_GTU1	IRQ_DAU	IRQ_CAU
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
IRQ_CAU	0	Interrupt set-enable bit for Current Acquisition Unit Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_DAU	1	Interrupt set-enable bit for Data Acquisition Unit Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU1	2	Interrupt set-enable bit for General Timer Unit1 Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled

		1 = interrupt enabled.
IRQ_GTU2	3	Interrupt set-enable bit for General Timer Unit2 Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU3	4	Interrupt set-enable bit for General Timer Unit3 Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU4	5	Interrupt set-enable bit for General Timer Unit4 Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU5	6	Interrupt set-enable bit for General Timer Unit5 Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU6	7	Interrupt set-enable bit for General Timer Unit6 Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU7	8	Interrupt set-enable bit for General Timer Unit7 Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.

IRQ_GTU8	9	Interrupt set-enable bit for General Timer Unit8 Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_SCI1	10	Interrupt set-enable bit for Serial Communication Unit1 Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_SCI2	11	Interrupt set-enable bit for Serial Communication Unit2 Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_LIN	12	Interrupt set-enable bit for LIN Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_Ig	13	Interrupt set-enable bit for Ignition Input Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled. Note: GDU automatic polling must be enabled otherwise triggered when polled.
IRQ_GDU_st	14	Interrupt set-enable bit for Gate Driver Unit Status Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.

		Note: GDU automatic polling must be enabled otherwise triggered when polled.
IRQ_CAN	15	Interrupt set-enable bit for CAN Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GPIO	16	Interrupt set-enable bit for GPIO Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_AMCT	17	Interrupt set-enable bit for Advanced Motor Control Timer Unit Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_DMA	18	Interrupt set-enable bit for DMA Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_SysCon	19	Interrupt set-enable bit for System Control Unit Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.

Interrupt Clear-enable Registers (NVIC_ICER)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E180

The NVIC_ICER registers disable interrupts, and show which interrupts are enabled.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24

-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	IRQ_SysCon	IRQ_DMA	IRQ_AMCT	IRQ_GPIO
U	U	U	U	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
IRQ_CAN	IRQ_GDU_st	IRQ _Ig	IRQ_LIN	IRQ_SCI2	IRQ_SCI1	IRQ_GTU8	IRQ_GTU7
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IRQ_GTU6	IRQ_GTU5	IRQ_GTU4	IRQ_GTU3	IRQ_GTU2	IRQ_GTU1	IRQ_DAU	IRQ_CAU
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
IRQ_CAU	0	Interrupt clear-enable bit for Current Acquisition Unit Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_DAU	1	Interrupt clear-enable bit for Data Acquisition Unit Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU1	2	Interrupt clear-enable bit for General Timer Unit1 Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU2	3	Interrupt clear-enable bit for General Timer Unit2 Write: 0 = no effect 1 = disable interrupt.

		Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU3	4	Interrupt clear-enable bit for General Timer Unit3 Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU4	5	Interrupt clear-enable bit for General Timer Unit4 Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU5	6	Interrupt clear-enable bit for General Timer Unit5 Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU6	7	Interrupt clear-enable bit for General Timer Unit6 Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU7	8	Interrupt clear-enable bit for General Timer Unit7 Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GTU8	9	Interrupt clear-enable bit for General Timer Unit8 Write: 0 = no effect 1 = disable interrupt. Read:

		0 = interrupt disabled 1 = interrupt enabled.
IRQ_SCI1	10	Interrupt clear-enable bit for Serial Communication Unit1 Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_SCI2	11	Interrupt clear-enable bit for Serial Communication Unit2 Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_LIN	12	Interrupt clear-enable bit for LIN Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_Ig	13	Interrupt clear-enable bit for Ignition Input Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled. Note: GDU automatic polling must be enabled otherwise triggered when polled.
IRQ_GDU_st	14	Interrupt clear-enable bit for Gate Driver Unit Status Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled. Note: GDU automatic polling must be enabled otherwise triggered when polled.
IRQ_CAN	15	Interrupt clear-enable bit for CAN Write: 0 = no effect

		1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_GPIO	16	Interrupt clear-enable bit for GPIO Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_AMCT	17	Interrupt clear-enable bit for Advanced Motor Control Timer Unit Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_DMA	18	Interrupt clear-enable bit for DMA Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.
IRQ_SysCon	19	Interrupt clear-enable bit for System Control Unit Write: 0 = no effect 1 = disable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.

Interrupt Set-pending Registers (NVIC_ISPR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E200

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	IRQ_SysCon	IRQ_DMA	IRQ_AMCT	IRQ_GPIO
U	U	U	U	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
IRQ_CAN	IRQ_GDU_st	IRQ _lg	IRQ_LIN	IRQ_SCI2	IRQ_SCI1	IRQ_GTU8	IRQ_GTU7
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IRQ_GTU6	IRQ_GTU5	IRQ_GTU4	IRQ_GTU3	IRQ_GTU2	IRQ_GTU1	IRQ_DAU	IRQ_CAU
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
IRQ_CAU	0	Interrupt set-pending bits for Current Acquisition Unit Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_DAU	1	Interrupt set-pending bits for Data Acquisition Unit Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU1	2	Interrupt set-pending bits for General Timer Unit1 Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU2	3	Interrupt set-pending bits for General Timer Unit2 Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU3	4	Interrupt set-pending bits for General Timer Unit3 Write: 0 = no effect

		1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU4	5	Interrupt set-pending bits for General Timer Unit4 Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU5	6	Interrupt set-pending bits for General Timer Unit5 Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU6	7	Interrupt set-pending bits for General Timer Unit6 Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU7	8	Interrupt set-pending bits for General Timer Unit7 Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU8	9	Interrupt set-pending bits for General Timer Unit8 Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_SCI1	10	Interrupt set-pending bits for Serial Communication Unit1 Write: 0 = no effect 1 = changes interrupt state to pending.

		Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_SCI2	11	Interrupt set-pending bits for Serial Communication Unit2 Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_LIN	12	Interrupt set-pending bits for LIN Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_Ig	13	Interrupt set-pending bits for Ignition Input Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending. Note: GDU automatic polling must be enabled otherwise triggered when polled.
IRQ_GDU_st	14	Interrupt set-pending bits for Gate Driver Unit Status Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending. Note: GDU automatic polling must be enabled otherwise triggered when polled.
IRQ_CAN	15	Interrupt set-pending bits for CAN Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GPIO	16	Interrupt set-pending bits for GPIO Write:

		0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_AMCT	17	Interrupt set-pending bits for Advanced Motor Control Timer Unit Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_DMA	18	Interrupt set-pending bits for DMA Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_SysCon	19	Interrupt set-pending bits for System Control Unit Write: 0 = no effect 1 = changes interrupt state to pending. Read: 0 = interrupt is not pending 1 = interrupt is pending.

Interrupt Clear-pending Registers (NVIC_ICPR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E280

The NVIC_ICPR registers remove the pending state from interrupts, and show which interrupts are pending.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	IRQ_SysCon	IRQ_DMA	IRQ_AMCT	IRQ_GPIO
U	U	U	U	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
IRQ_CAN	IRQ_GDU_st	IRQ _Ig	IRQ_LIN	IRQ_SCI2	IRQ_SCI1	IRQ_GTU8	IRQ_GTU7
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0

IRQ_GTU6	IRQ_GTU5	IRQ_GTU4	IRQ_GTU3	IRQ_GTU2	IRQ_GTU1	IRQ_DAU	IRQ_CAU
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
IRQ_CAU	0	Interrupt clear-pending bits for Current Acquisition Unit Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_DAU	1	Interrupt clear-pending bits for Data Acquisition Unit Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU1	2	Interrupt clear-pending bits for General Timer Unit1 Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU2	3	Interrupt clear-pending bits for General Timer Unit2 Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU3	4	Interrupt clear-pending bits for General Timer Unit3 Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU4	5	Interrupt clear-pending bits for General Timer Unit4

		Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU5	6	Interrupt clear-pending bits for General Timer Unit5 Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU6	7	Interrupt clear-pending bits for General Timer Unit6 Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU7	8	Interrupt clear-pending bits for General Timer Unit7 Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GTU8	9	Interrupt clear-pending bits for General Timer Unit8 Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_SCI1	10	Interrupt clear-pending bits for Serial Communication Unit1 Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_SCI2	11	Interrupt clear-pending bits for Serial Communication Unit2 Write:

		0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_LIN	12	Interrupt clear-pending bits for LIN Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_Ig	13	Interrupt clear-pending bits for Ignition Input Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending. Note: GDU automatic polling must be enabled otherwise triggered when polled.
IRQ_GDU_st	14	Interrupt clear-pending bits for Gate Driver Unit Status Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending. Note: GDU automatic polling must be enabled otherwise triggered when polled.
IRQ_CAN	15	Interrupt clear-pending bits for CAN Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_GPIO	16	Interrupt clear-pending bits for GPIO Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.

IRQ_AMCT	17	Interrupt clear-pending bits for Advanced Motor Control Timer Unit Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_DMA	18	Interrupt clear-pending bits for DMA Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.
IRQ_SysCon	19	Interrupt clear-pending bits for System Control Unit Write: 0 = no effect 1 = removes pending state an interrupt. Read: 0 = interrupt is not pending 1 = interrupt is pending.

Writing 1 to an ICPR bit does not affect the active state of the corresponding interrupt.

Interrupt Active Bit Registers (NVIC_IABR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E300

The NVIC_IABR registers indicate which interrupts are active.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	IRQ_SysCon	IRQ_DMA	IRQ_AMCT	IRQ_GPIO
U	U	U	U	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
IRQ_CAN	IRQ_GDU_st	IRQ _Ig	IRQ_LIN	IRQ_SCI2	IRQ_SCI1	IRQ_GTU8	IRQ_GTU7
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IRQ_GTU6	IRQ_GTU5	IRQ_GTU4	IRQ_GTU3	IRQ_GTU2	IRQ_GTU1	IRQ_DAU	IRQ_CAU
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:

R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
IRQ_CAU	0	Interrupt active flags for Current Acquisition Unit 0 = interrupt not active 1 = interrupt active.
IRQ_DAU	1	Interrupt active flags for Data Acquisition Unit 0 = interrupt not active 1 = interrupt active.
IRQ_GTU1	2	Interrupt active flags for General Timer Unit1 0 = interrupt not active 1 = interrupt active.
IRQ_GTU2	3	Interrupt active flags for General Timer Unit2 0 = interrupt not active 1 = interrupt active.
IRQ_GTU3	4	Interrupt active flags for General Timer Unit3 0 = interrupt not active 1 = interrupt active.
IRQ_GTU4	5	Interrupt active flags for General Timer Unit4 0 = interrupt not active 1 = interrupt active.
IRQ_GTU5	6	Interrupt active flags for General Timer Unit5 0 = interrupt not active 1 = interrupt active.
IRQ_GTU6	7	Interrupt active flags for General Timer Unit6 0 = interrupt not active 1 = interrupt active.
IRQ_GTU7	8	Interrupt active flags for General Timer Unit7 0 = interrupt not active 1 = interrupt active.
IRQ_GTU8	9	Interrupt active flags for General Timer Unit8 0 = interrupt not active 1 = interrupt active.
IRQ_SCI1	10	Interrupt active flags for Serial Communication Unit1 0 = interrupt not active 1 = interrupt active.
IRQ_SCI2	11	Interrupt active flags for Serial Communication Unit2 0 = interrupt not active 1 = interrupt active.
IRQ_LIN	12	Interrupt active flags for LIN 0 = interrupt not active 1 = interrupt active.

IRQ_Ig	13	Interrupt active flags for Ignition Input 0 = interrupt not active 1 = interrupt active. Note: GDU automatic polling must be enabled otherwise triggered when polled.
IRQ_GDU_st	14	Interrupt active flags for Gate Driver Unit Status 0 = interrupt not active 1 = interrupt active.Note: GDU automatic polling must be enabled otherwise triggered when polled.
IRQ_CAN	15	Interrupt active flags bit for CAN 0 = interrupt not active 1 = interrupt active.
IRQ_GPIO	16	Interrupt active flags for GPIO 0 = interrupt not active 1 = interrupt active.
IRQ_AMCT	17	Interrupt active flags for Advanced Motor Control Timer Unit 0 = interrupt not active 1 = interrupt active.
IRQ_DMA	18	Interrupt active flags for DMA 0 = interrupt not active 1 = interrupt active.
IRQ_SysCon	19	Interrupt active flags for System Control Unit 0 = interrupt not active 1 = interrupt active.

A bit reads as one if the status of the corresponding interrupt is active or active and pending.

Interrupt Priority Registers (NVIC_IPR0)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E400

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
IRQ_GTU2							
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
IRQ_GTU1							
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
IRQ_DAU							
U	U	U	U	U	U		

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IRQ_CAU							
U	U	U	U	U			

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
IRQ_CAU	7:0	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_DAU	15:8	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_GTU1	23:16	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_GTU2	31:24	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.

Interrupt Priority Registers (NVIC_IPR1)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E404

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
IRQ_GTU6							
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
IRQ_GTU5							
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
IRQ_GTU4							
U	U	U	U	U	U		

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IRQ_GTU3							
U	U	U	U	U			

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
IRQ_GTU3	7:0	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_GTU4	15:8	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.

IRQ_GTU5	23:16	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_GTU6	31:24	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.

Interrupt Priority Registers (NVIC_IPR2)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E408

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
IRQ_SCI2							
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
IRQ_SCI1							
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
IRQ_GTU8							
U	U	U	U	U	U		

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IRQ_GTU7							
U	U	U	U	U			

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
IRQ_GTU7	7:0	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_GTU8	15:8	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_SCI1	23:16	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_SCI2	31:24	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.

Interrupt Priority Registers (NVIC_IPR3)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E40C

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24

IRQ_CAN							
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
IRQ_GDU_st							
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
IRQ _lg							
U	U	U	U	U	U		

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IRQ_LIN							
U	U	U	U	U			

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
IRQ_LIN	7:0	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ _lg	15:8	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_GDU_st	23:16	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_CAN	31:24	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.

Interrupt Priority Registers (NVIC_IPR4)

Reset value = 0x0000_0000 RegisterAddress: 0xE000E410

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
IRQ_SysCon							
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
IRQ_DMA							
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
IRQ_AMCT							
U	U	U	U	U	U		

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IRQ_GPIO							
U	U	U	U	U			

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
IRQ_GPIO	7:0	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_AMCT	15:8	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_DMA	23:16	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
IRQ_SysCon	31:24	Each implementation-defined priority field can hold a priority value, 0-8. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.

Software Trigger Interrupt Register (STIR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000EF00

Write to the STIR to generate an interrupt from software.

When the USERSETMPEND bit in the SCR is set to 1, unprivileged software can access the STIR.

Note: Only privileged software can enable unprivileged access to the STIR.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-		
U	U	U	U	U	U		

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
INTID[7:0]							
W	W	W	W	W	W	W	W

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description

INTID	7:0	Interrupt ID of the interrupt to trigger, in the range 0-239. For example, a value of 0x03 specifies interrupt IRQ3.

3.3.3.1 Level-sensitive and pulse interrupts

A Cortex-M4 device can support both level-sensitive and pulse interrupts. Pulse interrupts are also described as edge-triggered interrupts.

A level-sensitive interrupt is held asserted until the peripheral deasserts the interrupt signal. Typically this happens because the ISR accesses the peripheral, causing it to clear the interrupt request. A pulse interrupt is an interrupt signal sampled synchronously on the rising edge of the processor clock. To ensure the NVIC detects the interrupt, the peripheral must assert the interrupt signal for at least one clock cycle, during which the NVIC detects the pulse and latches the interrupt.

When the processor enters the ISR, it automatically removes the pending state from the interrupt, see Hardware and software control of interrupts. For a level-sensitive interrupt, if the signal is not deasserted before the processor returns from the ISR, the interrupt becomes pending again, and the processor must execute its ISR again. This means that the peripheral can hold the interrupt signal asserted until it no longer requires servicing.

3.3.3.2 Hardware and software control of interrupts

The Cortex-M4 latches all interrupts. A peripheral interrupt becomes pending for one of the following reasons:

- the NVIC detects that the interrupt signal is HIGH and the interrupt is not active
- the NVIC detects a rising edge on the interrupt signal
- software writes to the corresponding interrupt set-pending register bit, see Interrupt

Set-pending Registers, or to the STIR to make an interrupt pending.

A pending interrupt remains pending until one of the following:

- The processor enters the ISR for the interrupt. This changes the state of the interrupt from pending to active. Then:
  - For a level-sensitive interrupt, when the processor returns from the ISR, the NVIC samples the interrupt signal. If the signal is asserted, the state of the interrupt changes to pending, which might cause the processor to immediately re-enter the ISR. Otherwise, the state of the interrupt changes to inactive.
  - For a pulse interrupt, the NVIC continues to monitor the interrupt signal, and if this is pulsed the state of the interrupt changes to pending and active. In this case, when the processor returns from the ISR the state of the interrupt changes to pending, which might cause the processor to immediately re-enter the ISR.
    If the interrupt signal is not pulsed while the processor is in the ISR, when the processor returns from the ISR the state of the interrupt changes to inactive.
- Software writes to the corresponding interrupt clear-pending register bit.
  For a level-sensitive interrupt, if the interrupt signal is still asserted, the state of the interrupt does not change. Otherwise, the state of the interrupt changes to inactive.

  For a pulse interrupt, state of the interrupt changes to:

  - inactive, if the state was pending
  - active, if the state was active and pending.

3.3.4 Memory Protection Unit

This section describes the optional Memory Protection Unit (MPU).

The MPU divides the memory map into a number of regions, and defines the location, size, access permissions, and memory attributes of each region. It supports:

- independent attribute settings for each region
- overlapping regions
- export of memory attributes to the system.

The memory attributes affect the behavior of memory accesses to the region. The Cortex-M4

MPU defines:

- eight separate memory regions, 0-7
- a background region.

When memory regions overlap, a memory access is affected by the attributes of the region with the highest number. For example, the attributes for region 7 take precedence over the attributes of any region that overlaps region 7.

The background region has the same memory access attributes as the default memory map, but is accessible from privileged software only.

The Cortex-M4 MPU memory map is unified. This means instruction accesses and data accesses have same region settings.

If a program accesses a memory location that is prohibited by the MPU, the processor generates a MemManage fault. This causes a fault exception, and might cause termination of the process in an OS environment. In an OS environment, the kernel can update the MPU region setting dynamically based on the process to be executed. Typically, an embedded OS uses the MPU for memory protection.

Configuration of MPU regions is based on memory types, see Memory regions, types and attributes.

Table 4-37 shows the possible MPU region attributes. These include Shareability and cache behavior attributes that are not relevant to most microcontroller implementations. See MPU configuration for a microcontroller and your vendor documentation for programming guidelines if implemented.

Memory type	Shareability	Other attributes	Description
Strongly- ordered	-	-	All accesses to Strongly-ordered memory occur in program order. All Strongly-ordered regions are assumed to be shared.
Device	Shared	-	Memory-mapped peripherals that several processors share.
	Non-shared	-	Memory-mapped peripherals that only a single processor uses.
Normal	Shared	Non-cacheable Write-through Cacheable Write-back Cacheable	Normal memory that is shared between several processors.
	Non-shared	Non-cacheable Write-through Cacheable Write-back Cacheable	Normal memory that only a single processor uses

Address	Name	Type	Required Previlege	Reset Value
0xE000_ED90	MPU_TYPE	RO	Privileged	0x0000_0800
0xE000_ED94	MPU_CTRL	RW	Privileged	0x0000_0000
0xE000_ED98	MPU_RNR	RW	Privileged	0x0000_0000
0xE000_ED9C	MPU_RBAR	RW	Privileged	0x0000_0000
0xE000_EDA0	MPU_RASR	RW	Privileged	0x0000_0000
0xE000_EDA4	MPU_RBAR_A1	RW	Privileged	0x0000_0000
0xE000_EDA8	MPU_RASR_A1	RW	Privileged	0x0000_0000
0xE000_EDAC	MPU_RBAR_A2	RW	Privileged	0x0000_0000
0xE000_EDB0	MPU_RASR_A2	RW	Privileged	0x0000_0000
0xE000_EDB4	MPU_RBAR_A3	RW	Privileged	0x0000_0000
0xE000_EDB8	MPU_RASR_A3	RW	Privileged	0x0000_0000

MPU Type Register (MPU_TYPE)

Reset value = 0x0000_0800 RegisterAddress: 0xE000_ED90

The MPU_TYPE register indicates whether the MPU is present, and if so, how many regions it supports. The bit assignments are:

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
IREGION							
R	R	R	R	R	R	R	R

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
DREGION							
R	R	R	R	R	R	R	R

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	-	-	-	-	SEPARATE
U	U	U	U	U	U	U	R

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
SEPARATE	0	Indicates support for unified or separate instruction and date memory maps: 0 = unified.
DREGION	15:8	Indicates the number of supported MPU data regions: 0x08 = Eight MPU regions.
IREGION	23:16	Indicates the number of supported MPU instruction regions. Always contains 0x00. The MPU memory map is unified and is described by the DREGION field.

MPU Control Register (MPU_CTRL)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED94

The MPU_CTRL register:

• enables the MPU

• enables the default memory map background region

• enables use of the MPU when in the hard fault, Non-maskable Interrupt (NMI), and FAULTMASK escalated handlers.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-		
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	-	-	PRIVDEFENA	HFNMIENA	ENABLE
U	U	U	U	U	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
ENABLE	0	Enables the MPU: 0 = MPU disabled 1 = MPU enabled.
HFNMIENA	1	Enables the operation of MPU during hard fault, NMI, and FAULTMASK handlers. When the MPU is enabled: 0 = MPU is disabled during hard fault, NMI, and FAULTMASK handlers, regardless of the value of the ENABLE bit 1 = the MPU is enabled during hard fault, NMI, and FAULTMASK handlers. When the MPU is disabled, if this bit is set to 1 the behavior is Unpredictable.
PRIVDEFENA	2	Enables privileged software access to the default memory map: 0 = If the MPU is enabled, disables use of the default memory map. Any memory access to a location not covered by any enabled region causes a fault. 1 = If the MPU is enabled, enables use of the default memory map as a background region for privileged software accesses. When enabled, the background region acts as if it is region number -1. Any region that is defined and enabled has priority over this default map. If the MPU is disabled, the processor ignores this bit.

When ENABLE and PRIVDEFENA are both set to 1:

- For privileged accesses, the default memory map is as described in Memory model. Any access by privileged software that does not address an enabled memory region behaves as defined by the default memory map.
- Any access by unprivileged software that does not address an enabled memory region causes a MemManage fault.

XN and Strongly-ordered rules always apply to the System Control Space regardless of the value of the ENABLE bit.
When the ENABLE bit is set to 1, at least one region of the memory map must be enabled for the system to function unless the PRIVDEFENA bit is set to 1. If the PRIVDEFENA bit is set to 1 and no regions are enabled, then only privileged software can operate.
When the ENABLE bit is set to 0, the system uses the default memory map. This has the same memory attributes as if the MPU is not implemented. The default memory map applies to accesses from both privileged and unprivileged software.
When the MPU is enabled, accesses to the System Control Space and vector table are always permitted. Other areas are accessible based on regions and whether PRIVDEFENA is set to 1.

Unless HFNMIENA is set to 1, the MPU is not enabled when the processor is executing the handler for an exception with priority –1 or –2. These priorities are only possible when handling a hard fault or NMI exception, or when FAULTMASK is enabled. Setting the HFNMIENA bit to 1 enables the MPU when operating with these two priorities.

MPU Region Number Register (MPU_RNR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED98

The MPU_RNR selects which memory region is referenced by the MPU_RBAR and MPU_RASR registers.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-		
U	U	U	U	U	U		

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
REGION[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:	
R: Read	U: Unimplemented bit, read as 0
W: Write	Res: Reserved bit
1C: Write 1 to clear. Writing 0 has no effect	

Field	Bits	Description
REGION	7:0	Indicates the MPU region referenced by the MPU_RBAR and MPU_RASR registers. The MPU supports 8 memory regions, so the permitted values of this field are 0-7.

Normally, you write the required region number to this register before accessing the MPU_RBAR or MPU_RASR. However you can change the region number by writing to the MPU RBAR with the VALID bit set to 1. This write updates the value of the REGION field.

MPU Region Base Address Register (MPU_RBAR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED9C

The MPU_RBAR defines the base address of the MPU region selected by the MPU_RNR, and can update the value of the MPU_RNR.

Write MPU_RBAR with the VALID bit set to 1 to change the current region number and update the MPU_RNR. The bit assignments are:

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
ADDR							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16

ADDR							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
ADDR							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
ADDR			VALID	REGION[3:0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
REGION	3:0	MPU region field: For the behavior on writes, see the description of the VALID field. On reads, returns the current region number, as specified by the RNR.
VALID	4	MPU Region Number valid bit: Write: 0 = MPU_RNR not changed, and the processor: • updates the base address for the region specified in the MPU_RNR • ignores the value of the REGION field 1 = the processor: • updates the value of the MPU_RNR to the value of the REGION field • updates the base address for the region specified in the REGION field. Always reads as zero.
ADDR	31:N	Region base address field. The value of N depends on the region size. For more information see The ADDR field.

The ADDR field

The ADDR field is bits[31:N] of the MPU_RBAR. The region size, as specified by the SIZE field in the MPU_RASR, defines the value of N:
N = Log2(Region size in bytes),
If the region size is configured to 4GB, in the MPU_RASR, there is no valid ADDR field. In this case, the region occupies the complete memory map, and the base address is 0x00000000. The base address is aligned to the size of the region. For example, a 64KB region must be aligned on a multiple of 64KB, for example, at 0x00010000 or 0x00020000.

MPU Region Attribute and Size Register (MPU_RASR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000EDA0

The MPU_RASR defines the region size and memory attributes of the MPU region specified by the MPU_RNR, and enables that region and any subregions.

MPU_RASR is accessible using word or halfword accesses:

- the most significant halfword holds the region attributes
- the least significant halfword holds the region size and the region and subregion enable bits.

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	XN	-	AP		
U	U	U	R/W	U	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	TEX			S	C	B
U	U	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
SRD							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	SIZE					ENABLE
U	U	R/W	R/W	R/W	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
ENABLE	0	Region enable bit.
SIZE	5:1	Specifies the size of the MPU protection region. The minimum permitted value is 3 (0b00010). See SIZE field values for more information.
SRD	15:8	Subregion disable bits. For each bit in this field: 0 = corresponding sub-region is enabled 1 = corresponding sub-region is disabled See Subregions for more information. Region sizes of 128 bytes and less do not support subregions. When writing the attributes for such a region, write the SRD field as 0x00.
S	18	Shareable bit, see Table 3-3-4-1.
TEX, C, B	21:19, 17, 16	Memory access attributes, see Table 3-3-4-1.
AP	26:24	Access permission field, see Table3-3-4-3.
XN	28	Instruction access disable bit: 0 = instruction fetches enabled 1 = instruction fetches disabled.

SIZE field values

The SIZE field defines the size of the MPU memory region specified by the RNR. as follows:
(Region size in bytes) = 2(SIZE+1)
The smallest permitted region size is 32B, corresponding to a SIZE value of 4. Table 4-44 gives example SIZE values, with the corresponding region size and value of N in the MPU_RBAR.

Table: 3-3-4-1

SIZE Value	Region Size	Value of N	Note
0b00100 (4)	32B	5	Minimum permitted size
0b01001 (9)	1KB	10	-
0b10011 (19)	1MB	20	-
0b11101 (29)	1GB	30	-
0b11111 (31)	4GB	32	Maximum possible size

MPU access permission attributes

This section describes the MPU access permission attributes. The access permission bits, TEX, C, B, S, AP, and XN, of the RASR, control access to the corresponding memory region. If an access is made to an area of memory without the required permissions, then the MPU generates a permission fault. Table 3-3-4-1 shows encodings for the TEX, C, B, and S access permission bits.

Table: 3-3-4-1

TEX	C	B	S	Memory Type	Shareability	Other Attributes
0b000	0	0	x[a]	Strongly-ordered	Shareable	-
		1	x[a]	Device	Shareable	-
	1	0	0	Normal	Not Shareable	Outer and inner write-through. No write allocate.
			1		Shareable	
		1	0	Normal	Not Shareable	Outer and inner write-back. No write allocate.
			1		Shareable	
0b001	0	0	0	Normal	Not Shareable	
			1		Shareable	
		1	x[a]	Reserved encoding		-
	1	0	x[a]	Implementation defined attributes.		-
		1	0	Normal	Not Shareable	Outer and inner write-back. Write and read allocate.
			1		Shareable	Nonshared Device.
0b010	0	0	x[a]	Device	Not Shareable	Nonshared Device.
		1	x[a]	Reserved encoding		-
	1	x[a]	x[a]	Reserved encoding		-
0b1BB	A	A	0	Normal	Not Shareable	Cached memory, BB = outer policy, AA = inner policy. See Table 4-46 on page 4-44 for the encoding of the AA and BB bits.
			1		Shareable	

a. The MPU ignores the value of this bit.

Table 3-3-4-2 shows the cache policy for memory attribute encodings with a TEX value is in the range 4-7.

Table: 3-3-4-2

Encoding, AA or BB	Corresponding cache policy
00	Non-cacheable
01	Write back, write and read allocate
10	Write through, no write allocate
11	Write back, no write allocate

Table 3-3-4-3 shows the AP encodings that define the access permissions for privileged and unprivileged software.

Table: 3-3-4-3

AP[2:0]	Privileged permissions	Unprivileged permissions	Description
000	No access	No access	All accesses generate a permission fault
001	RW	No access	Access from privileged software only
010	RW	RO	Writes by unprivileged software generate a permission fault
011	RW	RW	Full access
100	Unpredictable	Unpredictable	Reserved
101	RO	No access	Reads by privileged software only
110	RO	RO	Read only, by privileged or unprivileged software
111	RO	RO	Read only, by privileged or unprivileged software

MPU mismatch

When an access violates the MPU permissions, the processor generates a MemManage fault. The MMFSR indicates the cause of the fault.

Updating an MPU region

To update the attributes for an MPU region, update the MPU_RNR, MPU_RBAR and MPU_RASR registers. You can program each register separately, or use a multiple-word write to program all of these registers. You can use the MPU_RBAR and MPU_RASR aliases to program up to four regions simultaneously using an STM instruction.

Updating an MPU region using separate words

Simple code to configure one region:

```
; R1 = region number
; R2 = size/enable
; R3 = attributes
; R4 = address
LDR R0,=MPU_RNR          ; 0xE000ED98, MPU region number register
STR R1, [R0, #0x0]       ; Region Number
STR R4, [R0, #0x4]       ; Region Base Address
STRH R2, [R0, #0x8]            ; Region Size and Enable
STRH R3, [R0, #0xA]            ; Region Attribute
```

Disable a region before writing new region settings to the MPU if you have previously enabled the region being changed. For example:

```
; R1 = region number
; R2 = size/enable
; R3 = attributes
; R4 = address
LDR R0,=MPU_RNR          ; 0xE000ED98, MPU region number register
STR R1, [R0, #0x0]       ; Region Number
BIC R2, R2, #1                 ; Disable
STRH R2, [R0, #0x8]      ; Region Size and Enable
STR R4, [R0, #0x4]       ; Region Base Address
STRH R3, [R0, #0xA]            ; Region Attribute
ORR R2, #1 ; Enable
STRH R2, [R0, #0x8]            ; Region Size and Enable
```

Software must use memory barrier instructions:

- before MPU setup if there might be outstanding memory transfers, such as buffered writes, that might be affected by the change in MPU settings
- after MPU setup if it includes memory transfers that must use the new MPU settings.

However, memory barrier instructions are not required if the MPU setup process starts by entering an exception handler, or is followed by an exception return, because the exception entry and exception return mechanism cause memory barrier behavior.

Software does not require any memory barrier instructions during MPU setup, because it accesses the MPU through the PPB, which is a Strongly-Ordered memory region.

For example, if you want all of the memory access behavior to take effect immediately after the programming sequence, use a DSB instruction and an ISB instruction. A DSB is required after changing MPU settings, such as at the end of context switch. An ISB is required if the code that programs the MPU region or regions is entered using a branch or call. If the programming sequence is entered using a return from exception, or by taking an exception, then you do not require an ISB.

Updating an MPU region using multi-word writes

You can program directly using multi-word writes, depending on how the information is divided. Consider the following reprogramming:

```
; R1 = region number
; R2 = address
; R3 = size, attributes in one
LDR R0, =MPU_RNR          ; 0xE000ED98, MPU region number register
STR R1, [R0, #0x0]        ; Region Number
STR R2, [R0, #0x4]        ; Region Base Address
STR R3, [R0, #0x8]        ; Region Attribute, Size and Enable
Use an STM instruction to optimize this:
; R1 = region number
; R2 = address
; R3 = size, attributes in one
LDR R0, =MPU_RNR          ; 0xE000ED98, MPU region number register
STM R0, {R1-R3}           ; Region Number, address, attribute, size and enable
```

You can do this in two words for pre-packed information. This means that the MPU_RBAR

contains the required region number and had the VALID bit set to 1. Use this when the data is statically packed, for example in a boot loader:

```
; R1 = address and region number in one
; R2 = size and attributes in one
LDR R0, =MPU_RBAR         ; 0xE000ED9C, MPU Region Base register
STR R1, [R0, #0x0]        ; Region base address and
; region number combined with VALID (bit 4) set to 1
STR R2, [R0, #0x4]        ; Region Attribute, Size and Enable
```

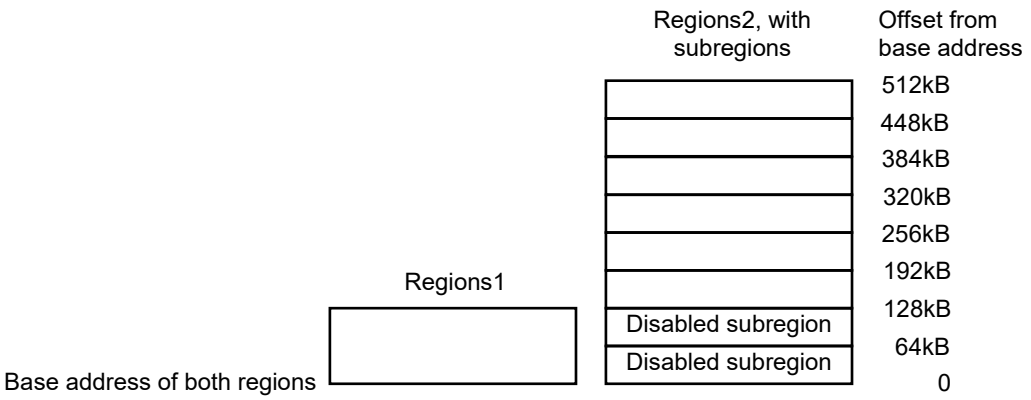
Subregions

Regions of 256 bytes or more are divided into eight equal-sized subregions. Set the corresponding bit in the SRD field of the MPU_RASR to disable a subregion. The least significant bit of SRD controls the first subregion, and the most significant bit controls the last subregion. Disabling a subregion means another region overlapping the disabled range matches instead. If no other enabled region overlaps the disabled subregion the MPU issues a fault.

Regions of 32, 64, and 128 bytes do not support subregions, With regions of these sizes, you must set the SRD field to 0x00, otherwise the MPU behavior is Unpredictable.

Example of SRD use

Two regions with the same base address overlap. Region one is 128KB, and region two is 512KB. To ensure the attributes from region one apply to the first 128KB region, set the SRD field for region two to 0b00000011 to disable the first two subregions, as the figure shows.



3.3.5 Floating Point Unit (FPU)

This section describes the optional Floating Point Unit (FPU) in a Cortex-M4F device. The FPU implements the FPv4-SP floating-point extension.

The FPU fully supports single-precision add, subtract, multiply, divide, multiply and accumulate, and square root operations. It also provides conversions between fixed-point and floating-point data formats, and floating-point constant instructions.

The FPU provides floating-point computation functionality that is compliant with the ANSI/IEEE Std 754-2008, IEEE Standard for Binary Floating-Point Arithmetic, referred to as the IEEE 754 standard.

The FPU contains 32 single-precision extension registers, which you can also access as 16 doubleword registers for load, store, and move operations.

Address	Name	Type	Reset Value
0xE000ED88	CPACR	RW	0x0000_0000
0xE000EF34	FPCCR	RW	0xC000_0000
0xE000EF38	FPCAR	RW	-
	FPSCR	RW	-
0xE000EF3C	FPDSCR	RW	0x0000_0000

Coprocessor Access Control Register (CPACR)

Reset value = 0x0000_0000 RegisterAddress: 0xE000ED88

The CPACR register specifies the access privileges for coprocessors. See the register summary in Cortex-M4F floating-point system registers for its attributes. The bit assignments are:

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	CP11	CP10	-	-	-	-
U	U	R/W	R/W	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-		
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	-	-			
U	U	U	U	U	U	U	U

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
CPn	[2n+1:2n] for n values10 and 11	Access privileges for coprocessor n. The possible values of each field are: 0b00 = Access denied. Any attempted access generates a NOCP UsageFault. 0b01 = Privileged access only. An unprivileged access generates a NOCP fault.

	0b10 = Reserved. The result of any access is Unpredictable. 0b11 = Full access.

Floating-point Context Control Register (FPCCR)

Reset value = 0xC000_0000 RegisterAddress: 0xE000EF34

The FPCCR register sets or returns FPU control data. See the register summary in Cortex-M4F floating-point system registers for its attributes. The bit assignments are:

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
ASPEN	LSPEN	-	-	-	-	-	-
R/W	R/W	U	U	U	U	U	U

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
-	-	-	-	-	-	-	-
U	U	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	MONRDY
U	U	U	U	U	U	U	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	BFRDY	MMRDY	HFRDY	THREAD	-	USER	LSPACT
U	R/W	R/W	R/W	R/W	U	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
LSPACT	0	0 = Lazy state preservation is not active. 1 = Lazy state preservation is active. floating-point stack frame has been allocated but saving state to it has been deferred.
USER	1	0 = Privilege level was not user when the floating-point stack frame was allocated. 1 = Privilege level was user when the floating-point stack frame was allocated.
THREAD	3	0 = Mode was not Thread Mode when the floating-point stack frame was allocated. 1 = Mode was Thread Mode when the floating-point stack frame was allocated.
HFRDY	4	0 = Priority did not permit setting the HardFault handler to the pending state when the floating-point stack frame was allocated. 1 = Priority permitted setting the HardFault handler to the pending state when the floating-point stack frame was allocated.
MMRDY	5	0 = MemManage is disabled or priority did not permit setting the MemManage handler to the pending state when the floating-point stack frame was allocated.

		1 = MemManage is enabled and priority permitted setting the MemManage handler to the pending state when the floating-point stack frame was allocated.
BFRDY	6	0 = BusFault is disabled or priority did not permit setting the BusFault handler to the pending state when the floating-point stack frame was allocated. 1 = BusFault is enabled and priority permitted setting the BusFault handler to the pending state when the floating-point stack frame was allocated.
MONRDY	8	0 = DebugMonitor is disabled or priority did not permit setting MON_PEND when the floating-point stack frame was allocated. 1 = DebugMonitor is enabled and priority permits setting MON_PEND when the floating-point stack frame was allocated.
LSPEN	30	0 = Disable automatic lazy state preservation for floating-point context. 1 = Enable automatic lazy state preservation for floating-point context.
ASPEN	31	Enables CONTROL<2> setting on execution of a floating-point instruction. This results in automatic hardware state preservation and restoration, for floating-point context, on exception entry and exit. 0 = Disable CONTROL<2> setting on execution of a floating-point instruction. 1 = Enable CONTROL<2> setting on execution of a floating-point instruction.

Floating-point Context Address Register (FPCAR)

Reset value: - RegisterAddress: 0xE000EF38

The FPCAR register holds the location of the unpopulated floating-point register space allocated on an exception stack frame. See the register summary in Cortex-M4F floating-point system registers for its attributes. The bit assignments are:

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
ADDRESS							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
ADDRESS							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
ADDRESS							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
ADDRESS							
R/W	R/W	R/W	R/W	R/W			

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description

ADDRESS	31:3	The location of the unpopulated floating-point register space allocated on an exception stack frame.

Floating-point Status Control Register (FPSCR)

Reset value = 0x0000_0000 RegisterAddress: -

The FPCAR register holds the location of the unpopulated floating-point register space allocated on an exception stack frame. See the register summary in Cortex-M4F floating-point system registers for its attributes. The bit assignments are:

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
N	Z	C	V	-	AHP	DN	FZ
R/W	R/W	R/W	R/W	U	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
RMode		-	-	-	-	-	-
R/W	R/W	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-		
U	U	U	U	U	U		

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
IDC	-	-	IXC	UFC	OFC	DZC	IOC
R/W	U	U	R/W	R/W	R/W	R/W	R/W

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
IOC	0	Cumulative exception bits for floating-point exceptions, see also bit [7]. Each of these bits is set to 1 to indicate that the corresponding exception has occurred since 0 was last written to it.
DZC	1	
OFC	2	
UFC	3	
IXC	4	
		IDC, bit[7] Input Denormal cumulative exception bit.
		IXC Inexact cumulative exception bit.
		UFC Underflow cumulative exception bit.
		OFC Overflow cumulative exception bit.
		DZC Division by Zero cumulative exception bit.
		IOC Invalid Operation cumulative exception bit.
IDC	7	Input Denormal cumulative exception bit, see bits [4:0].
RMode	23:22	Rounding Mode control field. The encoding of this field is: 0b00 Round to Nearest (RN) mode 0b01 Round towards Plus Infinity (RP) mode 0b10 Round towards Minus Infinity (RM) mode 0b11 Round towards Zero (RZ) mode. The specified rounding mode is used by almost all floating-point instructions.

FZ	24	Flush-to-zero mode control bit: 0: Flush-to-zero mode disabled. Behavior of the floating-point system is fully compliant with the IEEE 754 standard. 1: Flush-to-zero mode enabled.
DN	25	Default NaN mode control bit: 0 NaN operands propagate through to the output of a floating-point operation. 1 Any operation involving one or more NaNs returns the Default NaN.
AHP	26	Alternative half-precision control bit: 0 IEEE half-precision format selected. 1 Alternative half-precision format selected.
V	28	Condition code flags. Floating-point comparison operations update these flags. N Negative condition code flag. Z Zero condition code flag. C Carry condition code flag. V Overflow condition code flag.
C	29	
Z	30	
N	31	

Floating-point default status control register (FPCAR)

Reset value: 0x0000_0000 RegisterAddress: 0xE000EF3C

The FPDSCR register holds the default values for the floating-point status control data. See the register summary in Cortex-M4F floating-point system registers for its attributes.

The bit assignments are:

Bit31	Bit30	Bit29	Bit28	Bit27	Bit26	Bit25	Bit24
					AHP	DN	FZ
U	U	U	U	U	R/W	R/W	R/W

Bit23	Bit22	Bit21	Bit20	Bit19	Bit18	Bit17	Bit16
RMode							
R/W	R/W	U	U	U	U	U	U

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
U	U	U	U	U	U	U	U

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
U	U	U	U	U	U	U	U

Legend:
R: Read
W: Write
1C: Write 1 to clear. Writing 0 has no effect
U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description

RMode	23:22	Default value for FPSCR.RMode
FZ	24	Default value for FPSCR.FZ
DN	25	Default value for FPSCR.DN
AHP	26	Default value for FPSCR.AHP

Enabling the FPU
The FPU is disabled from reset. You must enable it before you can use any floating-point instructions. Following example shows an example code sequence for enabling the FPU in both privileged and user modes. The processor must be in privileged mode to read from and write to the CPACR.

```
; CPACR is located at address 0xE000ED88
LDR.W R0, =0xE000ED88
; Read CPACR
LDR R1, [R0]
; Set bits 20-23 to enable CP10 and CP11 coprocessors
ORR R1, R1, #(0xF << 20)
; Write back the modified value to the CPACR
STR R1, [R0]; wait for store to complete
DSB
;reset pipeline now the FPU is enabled
ISB
```

3.4 Bit Banding

Bit-banding is an optional feature of the Cortex-M4 processor. Bit-banding maps a complete word of memory onto a single bit in the bit-band region. For example, writing to one of the alias words sets or clears the corresponding bit in the bit-band region. This enables every individual bit in the bit-banding region to be directly accessible from a word-aligned address using a single LDR instruction. It also enables individual bits to be toggled without performing a read-modify-write sequence of instructions.

The processor memory map includes two bit-band regions. These occupy the lowest 1MB of the SRAM and Peripheral memory regions respectively. These bit-band regions map each word in an alias region of memory to a bit in a bit-band region of memory.

The System bus interface contains logic that controls bit-band accesses as follows:

- It remaps bit-band alias addresses to the bit-band region.
- For reads, it extracts the requested bit from the read byte, and returns this in the Least Significant Bit (LSB) of the read data returned to the core.
- For writes, it converts the write to an atomic read-modify-write operation.
- The processor does not stall during bit-band operations unless it attempts to access the System bus while the bit-band operation is being carried out.

The memory map has two 32-MB alias regions that map to two 1-MB bit-band regions:

- Accesses to the 32-MB SRAM alias region map to the 1-MB SRAM bit-band region.
- Accesses to the 32-MB peripheral alias region map to the 1-MB peripheral bit-band region.

A mapping formula shows how to reference each word in the alias region to a corresponding bit, or target bit, in the bit-band region. The mapping formula is:

bit_word_offset = (byte_offset x 32) + (bit_number × 4)

bit_word_addr = bit_band_base + bit_word_offset

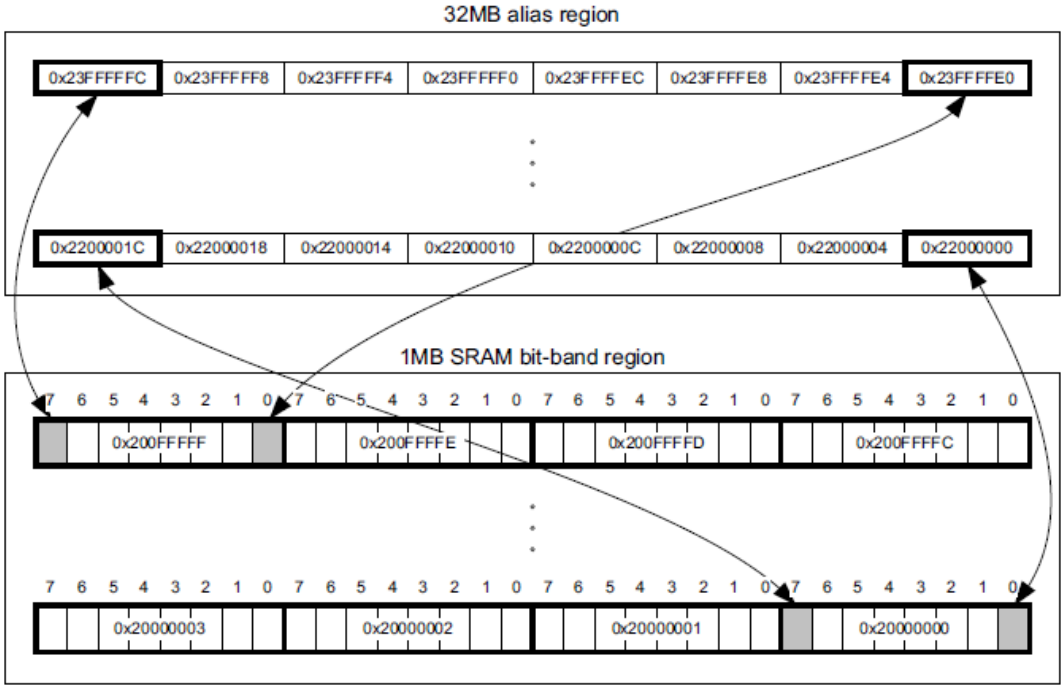
where:

- bit_word_offset is the position of the target bit in the bit-band memory region.
- bit_word_addr is the address of the word in the alias memory region that maps to the targeted bit.
- bit_band_base is the starting address of the alias region.
- byte_offset is the number of the byte in the bit-band region that contains the targeted bit.
- bit_number is the bit position, 0 to 7, of the targeted bit.

Figure 3-4-1 shows examples of bit-band mapping between the SRAM bit-band alias region and the SRAM bit-band region:

- The alias word at 0x23FFFFE0 maps to bit [0] of the bit-band byte at 0x200FFFFF: 0x23FFFFE0 = 0x22000000 + (0xFFFFF*32) + 0*4.
- The alias word at 0x23FFFFFC maps to bit [7] of the bit-band byte at 0x200FFFFF: 0x23FFFFFC = 0x22000000 + (0xFFFFF*32) + 7*4.
- The alias word at 0x22000000 maps to bit [0] of the bit-band byte at 0x20000000: 0x22000000 = 0x22000000 + (0*32) + 0*4.
- The alias word at 0x2200001C maps to bit [7] of the bit-band byte at 0x20000000: 0x2200001C = 0x22000000 + (0*32) + 7*4.

Figure 3-4-1



3.4.1 Directly accessing an alias region

Writing to a word in the alias region has the same effect as a read-modify-write operation on the targeted bit in the bit-band region.

Bit [0] of the value written to a word in the alias region determines the value written to the targeted bit in the bit-band region. Writing a value with bit [0] set writes a 1 to the bit-band bit, and writing a value with bit [0] cleared writes a 0 to the bit-band bit.

Bits [31:1] of the alias word have no effect on the bit-band bit. Writing 0x01 has the same effect as writing 0xFF. Writing 0x00 has the same effect as writing 0x0E.

Reading a word in the alias region returns either 0x01 or 0x00. A value of 0x01 indicates that the targeted bit in the bit-band region is set. A value of 0x00 indicates that the targeted bit is clear. Bits [31:1] are zero.

3.4.2 Directly accessing a bit-band region

You can directly access the bit-band region with normal reads and writes to that region.

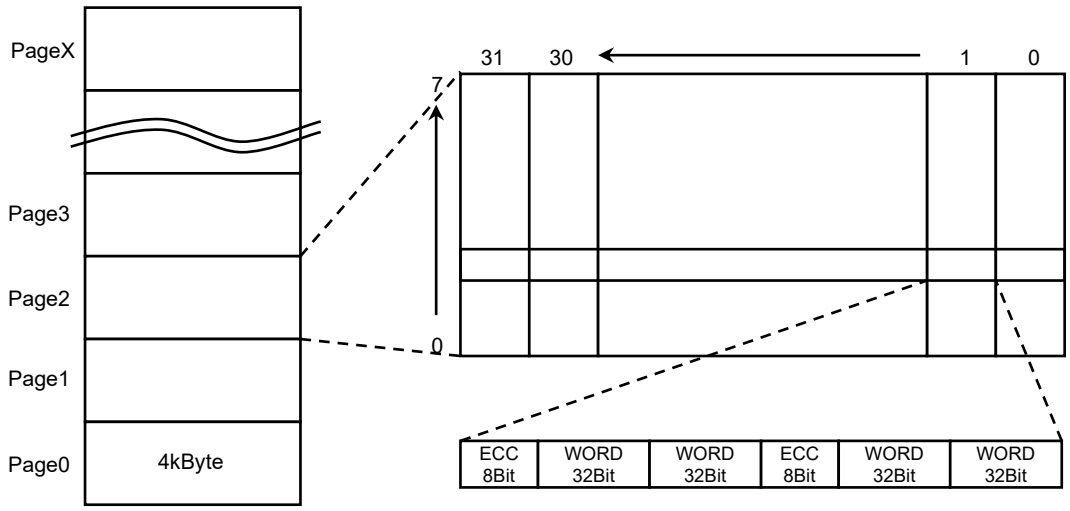
4.0 MEMORY

4.1 Flash Memory

A89201 has non-volatile memory, which is Flash memory. The size of Flash memory is available up to 128kbyte. The Flash memory can contain instructions as well as data. The use of Flash memory can be decided by programmer. The Flash sits on the NVM slave bus.

4.1.1 Memory architecture

Flash Memory has a page which contain 4kbyte for each. The page contain 8x31 section. Each single section contain 4 word and two ECC bit. The summary is described in below.



Flash Memory cell map

4.1.2 Programming

A89201 Flash memory can be programmed though SWD. The Allegro provided flash programmer shall be used when the code is programmed though SWD. Please refer the start-up manual for the SWD setup.

If the write command is inserted from AMBA bus to Flash area, it will generate a bus fault. The Flash program must though the program interfaces.

4.1.3 Prefetch

A89201 has a prefetch feature in order to minimize the flash memory read delay.

Using highest CPU clock speed, the Flash memory require 2 cycle to read out instruction from Flash memory. So CPU need to wait a clock cycle to fetch an instruction from Flash memory. Code speed will be degraded there is no prefetch feature.

A89201 Flash memory contain 144bits in a signle Flash address, which means more than 1 instruction will read out in a single read. When a CPU execution requires sequential instruction read, there will be no delay inserted.

CPU access to Flash memory will be done from AMBA instruction bus and prefetch block fetch the code as required and access speed is adjusted automatically.

4.1.4 ECC (Error Checking and Correction) on Flash

Flash memory store ECC for each instruction code. When single error is detected, the code will be automatically adjusted and report it to system control unit. When double error is detected, the correction won't be made and generate NMI interrupt.

The error checking is executed only when Flash access is made.

For fault reporting, please refer the system control unit.

4.2 DRAM (Data RAM)

A89201 has 32kbyte of DRAM. The DRAM has no access time penalty. The DRAM access will be completed within 1 clock cycle at any configuration if the access is granted. DRAM bus matrix sits on SRAM slave bus.

4.2.1 ECC (Error Checking and Correction) on DRAM

DRAM is protected by ECC. Single error will be corrected automatically and double error will be detected.

When single error is detected, ECC block only report to system control unit. If System control set-up, System Control unit IRQ can be flagged.

When double error is detected, ECC block report to system control unit. If System control set-up, System Control unit IRQ can be flagged.

4.3 IRAM (Instruction RAM)

A89201 has 8kbyte of IRAM. The IRAM has no access time penalty. The IRAM access will be completed within 1 clock cycle at any configuration when the access is granted. DRAM bus matrix sits on NVM slave bus.

4.3.1 ECC (Error Checking and Correction) on IRAM

IRAM is protected by ECC. Single error will be corrected automatically and double error will be detected.

When single error is detected, ECC block only report to system control unit. If System control set-up, System Control unit IRQ can be flagged.

When double error is detected, ECC block report to system control unit and NMI will be triggered. If System control set-up, System Control unit IRQ can be flagged.

4.4 BootROM

A89201 has 32kbyte BootROM contain the in-house initialization purpose. When the device is powered up, BootROM will be serviced and jump to user flash code address “0x0800_0000”. The BootROM sits on the NVM slave bus.

4.4.1 BIST(Build-in-self-test)

BootROM has BIST. When the device is powered up, CRC test will be executed in order to make sure the BootROM contents is correct. When the BIST is failed, the device will be held in reset until debugger is connected.

Revision Control

Aug 05/2021	Initial Revision
Nov 11/2021	Updated Logo Added Revision Control
Feb/19/2025	Removed bootloader Added note for peripheral access type Corrected bit position for parameter 'SEVONPEND' Corrected address offset for register STIR Corrected address offset for registers NVIC_IPR0-4 Corrected address offset and reset value for register Floating-point Context Control Register Corrected register name for Floating-point default status control register and its reset value Added jump address after bootROM execution